

우울증 예측 모델과 적용

- 간단한 우울증 진단 : 병원에 가지 않고 AI로 쉽게 우울증을 진단한다 -

2023년 5월

자카르타 한국 국제학교

남상혁

◇ 활동 계획서 ◇

연구자	학번 12304	이름 남상혁
논문 주제	우울증 예측 모델과 그것의 적용	
주제 선정 이유 및 목적	최근 뉴스 기사나 유튜브 영상을 보다보면 한국의 자살률이 엄청 높다는 내용들이 자주 보인다. 그래서 우리나라의 자살률이 얼마나 높은 지에 대해 조사를 하다보니, 세계보건기구의 조사에 따르면 대한민국은 OECD 국가들중 자살률이 총 4등인 것으로 나타났다. 또한 10대부터 30대까지의 사망원인중 가장 비중이 큰 것으로 나타났다. 심지어 이 수치는 계속해서 매년마다 증가하는 것으로 나타났는데, 도대체 왜 이렇게 자살률이 높은지에 대해 궁금증이 생기게 되었고, 이에 대해 조사를 해본 결과, 보건복지부에 조사에 따르면 우울증이 자살률과 상관관계가 제일 크다는 것에 대해 알수 있었다. 하지만 삼성서울병원의 연구에 따르면 사람들은 자기가 우울증에 걸린지 모르고, 특히 우리나라에서는 단순히 사춘기나 자신의 성격 탓으로 여기며 쉽게 넘어가며 정신건강의학과에 가서 체계적인 진료를 많이 받지 않는다. 또한 이러한 우울증은 날이 지날수록 점점 심해져 결국 극단적인 선택을 하게 만든다고 하는데, 그에 따라 나는 이러한 우울증을 보다 초기에 발견하여 증상이 심해지기전에 자신이 우울증인지 판단하는 AI를 만들고 그것을 직스 학생들 및 자카르타에 사는 한국인들을 상대로 적용하여 사람들에게 이 병에 대한 해결 방안이나 이 병에 대한 경각심을 다시 한 번 깨우칠 수 있게 할 것이다.	
연구방법	질병관리본부나 보건복지부와 같은 공식적인 기관에서 우울증에 관련한 데이터를 수집하여 이것에 대한 데이터를 분석하고, 그것을 데이터 시각화 프로그램인 matplotlib을 통해 시각화하여 어떤 것이 우울증에 제일 영향을 미치는 지도 조사를 하고, 해외 데이터 사이트인 kaggle이나 DBPIA와 같은 논문 사이트에 가서 관련 병에 대한 데이터를 수집한 후, 이것을 통해 로지스틱 회귀나 딥러닝으로 AI를 제작한 후, 검증을 하여 정확도를 높인 후, 만든 AI를 토대로 직스 학생들 및 인도네시아에 사는 한국인들에게 구글 폼으로 설문을 하여 직스 학생들이 얼마나 우울증에 걸렸는 지에 대해 탐구 한 후, 그들에게 우울증에 대한 해결 방안이나 이 병에 대한 경각심을 다시 한 번 깨우치게 할 것이다.	
예상 결과 및 효과	나는 내가 만든 AI를 통해 정신건강의학과에 가기전에 귀찮은 과정이나 절차를 거치지 않고, 또한 시간이 없어 병원에 가지 않아도 간단히 집에서 자신이 우울증에 걸렸는지 안걸렸는지에 대한 것을 간단하게 검사할 수 있는 우울증 예측 모델을 만들어서, 직스 학생들과 인도네시아에 사는 한국인들을 대상으로 설문을 진행하여 우울증을 예측하고자 한다. 나는 이 활동을 통해 사람들이 자신이 우울증에 걸린지 인지를 못하는 사람들에겐 우울증이 심해지기 전에 그에 대한 해결방법을 알려 줄 것이고, 우울증을 대수롭지 않게 여기는 사람들에게는 우울증에 대한 경각심을 심어줄 수 있다. 그리고 자신이 우울증이 걸렸는지 예측하는 AI를 통해 후에는 병원에 가지 않더라도 집과 학교에서도 우울증 진단을 할 수 있는 시스템으로 발전 할 수 있다.	
참고자료	서울 삼성병원 우울증과 자살 : http://www.samsunghospital.com/dept/medical/healthSub02View.do?content_id=1661&cPage=1&DP_CODE=DEP&MENU_ID=004025&ds_code=D0001673&main_content_id=406 자살률 증가 : https://www.hani.co.kr/arti/society/health/1046946.html OECD 자살률 순위: https://ko.wikipedia.org/wiki/OECD_%ED%9A%8C%EC%9B%90%EA%B5%AD_%EC%9E%90%EC%82%B4%EB%A5%A0_%EB%AA%A9%EB%A1%9D 블로그 인용한 글 : https://m.blog.naver.com/PostView.naver?isHttpsRedirect=true&blogId=iammaum&logNo=221221756690	

국문초록

주제어: 자살, 우울증, 예측, 치료, 인공지능(AI) - 딥러닝, 선형회귀

해가 지나면 지날수록, 국내에서는 점점 자살을 하는 사람이 늘어나고 있다. 보건복지부의 통계에 따르면 2020년에는 국내 자살사망자의 수는 1만 3195명이었던 것에 반해, 재작년인 2021년도에는 국내 자살사망자수가 1만 3352명으로 157명인 즉 1.2%나 증가한 것을 알 수 있었고, 이것은 OECD 국가들중 가장 자살률이 높으며 고령인구의 자살률도 높지만 특히 10대부터 30대까지의 사망원인중 가장 비중이 큰 것이 자살로 나타났다. 이것의 이유는 보건복지부의 조사에 따르면 정신건강 문제인 것으로 나타났고, 그중에서 가장 연관성이 높았던 우울증은 2017년도에는 우울증 환자수가 69만 1164명이었던 것에 반해, 재작년인 2021년도에는 93만 3481명으로 35.1%가 증가하였고, 매년 연평균 7.8%나 늘었다는 것을 확인할 수 있다. 이러한 우울증은 다른 정신병들과는 달리 대다수의 사람들은 자기가 우울증에 걸린지 모르고, 이것보다 더 문제인 것은 점점 심화가 되는 데 이것이 끝내는 극단적인 선택을 유발하게 된다고 한다. 우리는 이를 뉴스나 기사 아니면 주위 사람들을 통해 우울증에 대해서 알고는 있지만 정작 대부분의 사람들은 자신이 우울증에 걸리면, 우리나라에서는 대다수의 사람들이 자기가 우울증이 걸린 것을 부인하고, 그것이 단순히 사춘기나 자신의 성격 때문이라고 여기거나 정신건강의학과에 가는 것을 꺼리며 자신이 우울증에 걸렸다고 생각을 하지 않고 있는데 이것은 위험하다. 우울증은 자기 자신이 판단할 때는 자신이 우울증이라는 것을 알수 없기 때문에 병원에서의 객관적인 검사를 받고, 이후 약물 치료나 정신치료, 반복적 경두개 자기자극법을 통해 치료를 받아야한다. 하지만 대다수의 사람들은 이를 부끄러워하거나 단순히 쉽게 해결될 것이라고 여기며 간단히 넘긴다. 이에 본 논문에서는 데이터를 수집하여 우울증 예측 인공지능 모델을 만들고, 이를 실제로 우리가 현재 살고 있는 인도네시아에서 고향을 떠나 이곳에 와있는 한국 사람들과 우리 학교인 JIKS의 학생들 총 45명을 대상으로 진행한 설문 결과를 통해 우울증을 예측한다. 실제 우울증 환자 1503명의 데이터를 통해 딥러닝과 선형회귀를 활용하여 만든 우울증 예측 인공지능 프로그램중 더 정확도가 높았던 드롭아웃으로 과적합 문제를 해결하고, 최적화함수를 adam에서 RMS로 변환한 딥러닝 우울증 예측 AI모델이 약 90%의 정확도를 가지고 있었고, 오차는 0.23의 성능을 가지고 있다는 것을 확인할 수

있었다. 이 프로그램으로 우울증을 예측한 결과, 설문에 응해준 인도네시아의 45명의 사람들 중 29명이 우울증 의심 결과가 나왔다. 이 결과는 초기에 우울증 의심을 진단했다는 점에서 중요한 의미가 될수있었고, 또한 우울증 검사를 위해 굳이 병원에 찾아가지 않고 쉽게 우울증을 자가진단할 수 있다는 점에서 우울증에 대한 위험성을 각인 시켜주고, 이 병에 대한 인지율을 올릴수 있었다.

또한 추가적으로 조사해본 결과 45명의 설문대상자중 JIKS학생들이 17명, 나머지 다양한 연령대의 인도네시아의 한인들이 28명으로 나타났다. 그리고 JIKS학생들 17명중 11명인 64.7%가 우울증에 걸렸고, 나머지 연령대의 인도네시아의 한인들이 28명중 18명인 64.2%가 우울증에 걸린 것으로 나타났다. 이를 통해 둘의 차이는 그렇게 심하게 나지는 않지만 JIKS 학생들이 우울증 환자 비율이 더 높은 것을 알 수 있었다. 이러한 이유는 JIKS학생들 17명의 응답자 전원이 12학년이였었고, 그들을 대상으로 몇몇한테 질문을 해본 결과 많은 과제와 입시 스트레스 때문으로 나타났다. 이후 JIKS학생들이 얼마나 우울증에 대해 잘 알고 있는지에 대해 의문이 생겨 앙케이트를 실시한 결과, JIKS학생들중 많은 수의 학생이 우울증에 대해 잘 모르는 것으로 나타나였기에 우울증과 관련된 웹사이트와 포스터를 만든 후, JIKS에서 캠페인을 하며 학생들에게 우울증이라는 병에 대해 다시 한 번 경각심을 심어줄 수 있었다. 우리는 우리가 만든 우울증에 걸렸는지 예측하는 AI를 통해 시간이 없어, 병원에 가지 않더라도 집에서 몇초만에 쉽게 우울증 진단을 할 수 있는 시스템으로 발전시킬수있다.

1. 서론

1.1. 연구 배경

2. 본론

2.1. 이론적 배경

2.1.1. 우울증의 정의 및 증상

2.1.2. 우울증 진단 방법

2.1.3. 우울증의 원인

2.1.4. 우울증 환자 증가 배경

2.1.5. 우울증 치료 방법 및 예방 방법

2.1.6. 로지스틱 회귀

2.1.7. 딥러닝

2.1.8. AI 모델에서 사용된 파라미터

2.2. 연구 진행

2.2.1. 데이터 수집

2.2.2. 우울증과 속성들 간의 상관관계

2.2.3. 우울증 예측 로지스틱 회귀, 딥러닝 모델의 제작 과정 & 비교

2.2.4. 직스 학생 및 주변 성인 대상 설문

2.2.5. 설문 조사한 데이터를 우리가 만든 딥러닝 모델에 적용해보기

2.3. 연구 결과

3. 결론

3.1. 연구 요약 및 의의

4. 참고 문헌

1. 서론

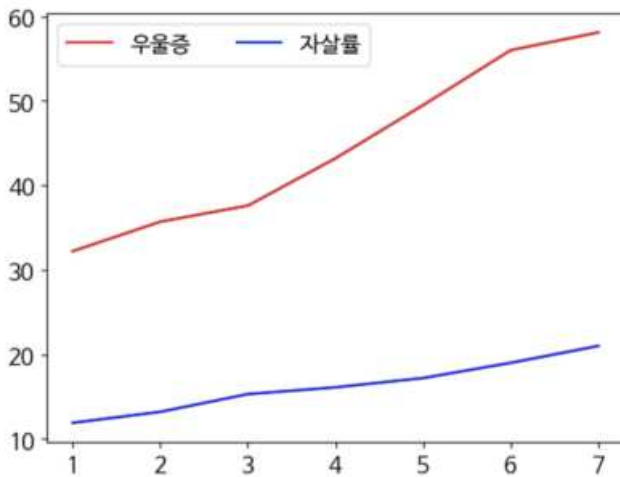
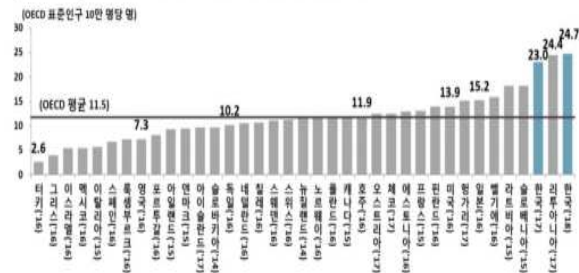
1.1. 연구 배경

날이 지나면 지날수록, 국내에서는 점점 자살을 하는 사람들이 엄청 늘어나고 있다. 보건복지부의 통계에 따르면, 자살사망자의 수는 2020년에는 1만 3195명이었던 것에 반해, 재작년인 2021년도에는 1만 3352명으로 157명인 즉 1.2%나 증가한 것을 알 수 있다. 또한 통계청의 조사에 따르면 특히 대한민국은 OECD 국가들중 자살률이 총 1등인 것으로 나타났고, 고령인구의 자살률도 높지만 특히 10대부터 30대까지의 사망원인중 가장 비중이 큰 것이 자살로 나타났다. 그리고 그것에 관한 이유는 보건복지부의 조사에 따르면 정신건강 문제라는 것을 밝혀내었는데, 그중 우울증이 제일 연관성이 높았고, 자살률과 함께 점점 수가 많이 늘고 있는 것으로 나타났다. 우울증은 건강보험심사평가원의 통계 결과에 따르면 우울증 환자 수가 2017년도에는 69만 1,164명이었던 것에 반해, 재작년인 2021년도에는 93만 3,481명으로 35.1%가 증가하였고, 매년 연평균 7.8% 증가했다는 것을



알 수 있다.

[그림 11] OECD 국가 연령표준화 자살률 비교



서울 삼성병원의 자료에 따르면 우울증은 다른 정신병들과는 달리 대다수의 사람들은 자기가 우울증에 걸린지 모르고, 이것보다 더 문제인 것은 점점 심화가 되는 데 이것이 끝내는 극단적인 선택을 유발하게 된다고 한다. 왼쪽의 그래프와 같이 우울증은 해가 갈수록 심화되어져 가고, 그 우울증이 심화되어져 갈수록, 자살률 또한 늘어나는 것을 확인할 수 있다. 그렇기에 우울증은 초기에

빠르게 병이 발생했는지를 확인하고 신속하게 대처하는 것이 중요하다. 하지만 우리나라에서는 대다수의 사람들이 자기가 우울증이 걸린 것을 부인하고, 그것이 단순히 사춘기나 자신의 성격 때문이라고 여기거나 정신건강의학과에 가는 것을 꺼려하거나 스스로 병을 해결할 수 있다고 생각하여 병원에 가서 체계적인 진료를 많이 받지 않는다. 또한 우울증을 검사하기 위해선 자기 스스로 병에 대해 판단하는 것이 어렵고, 병원에 가서 객관적인 검사를 필요로 하지만 대부분의 사람들이 이런 것을 거부하고, 병원에 가질

않거나 병원에 가더라도 병원에서 우울증이 나오는 것을 부끄러워하여 병원에서 하는 검사에 대해 거짓으로 검사 질문지를 작성하여 자신이 우울증이 아니라는 결과를 받게 된다. 그리고 이러면서 점점 우울증의 증상이 악화되고 나서 끝내 병원을 찾거나 끝까지 병원을 찾지 못하고 극단적인 선택을 하게 된다.



이에 본 논문은 앞서 언급했는 병원에서 부끄러운 검사를 거치지 않고, 객관적으로 검사결과를 내주고, 또한 시간이 없어 병원에 가지 않아도 간단히 집에서 자신이 우울증인지 아닌지를 검사할 수 있는 우울증 예측 인공지능 모델을 제안한다. 우리는 캐글(kaggle) 혹은 보건복지부이나 통계청 등의 사이트에서 1503명의 우울증 환자 데이터를 수집하여, 그 환자의 나이, 가끔 이유없는 슬픔을 느낄때가 있는지, 가족들에게 짜증을 내는지, 잠을 자는데 문제가 있는지, 집중을 잘 못하는지, 과식이나 적은 식욕을 가지고 있는지, 혹은 죄책감을 가지고 있는지, 가족과의 문제나 자살 시도와 같은 총 9개의 속성과 우울증의 양성, 음성 여부를 나타내는 각각 하나의 클래스를 포함하는 데이터를 통해 딥러닝과 선형회귀 모델을 사용하여 총 두가지의 AI를 만들어줄 것이다. 그런 후, 그 두가지 AI 모듈중 우리의 데이터에 더 적합한 AI 모듈을 찾아, 그 AI 모듈로, 한국을 벗어나 인도네시아에 살고 있는 자카르타 한국 국제학교(JIKS)의 학생들과 주변의 성인들을 대상으로 설문을 진행하여 우울증을 예측하고 그 데이터를 분석하고자 한다. 나는 이 활동을 통해 사람들이 우울증을 초기에 발견하지 못해 극단적인 선택을 하는 경우를 예방하고, 우울증을 가볍게 여기거나 이것을 별로 대수롭지 않게 여기던 사람들에게 경각심을 심어줄 수 있다. 또한, 우울증에 걸렸는지 예측하는 AI를 통해 병원에 가지 않더라도 집에서 쉽게 우울증 진단을 할 수 있는 시스템으로 발전 할 수 있다.

2. 본론

2.1. 이론적 배경

2.1.1. 우울증의 정의 및 증상

우울증은 서울대학교병원에 따르면, 의욕 저하와 우울감을 주요 증상으로 하여 다양한 인지 및 정신 신체적 증상을 일으켜 일상 기능의 저하를 가져오는 질환을 말한다. 우울장애는 평생 유병율이 **15%~25%** 정도에 이르며, 감정, 생각, 신체 상태, 그리고 행동 등에 변화를 일으키는 심각한 질환이다. 이것은 한 개인의 전반적인 삶에 영향을 준다. 우울증은 일시적인 우울감과는 다르며 개인적인 약함의 표현이거나 의지로 없앨 수 있는 것이 아니다. 상당수가 전문가의 도움을 받지 못하고 우울증으로 고생하는 경우가 많아 안타까운 질환이기도 하다. 그러나 전문가의 적절한 치료를 받는다면 상당한 호전을 기대할 수 있고 이전의 정상적인 생활로 돌아가는 것이 가능하다.

우울감과 삶에 대한 흥미 및 관심 상실이 우울증의 핵심 증상이다. 우울증의 가장 심각한 증상은 자살 사고로, 우울증 환자의 **2/3**에서 자살을 생각하고 **10~15%**에서 실제로 자살을 시행한다. 일부 우울증 환자는 자신이 우울증인 것을 알지 못하고 일상 생활에서 상당히 위축되어 기능이 떨어질 때까지도 자신의 기분 문제에 대해 호소하지 않는다. 또한, 우울증 환자의 **4/5** 정도가 수면 장애를 호소하는데 특히 아침까지 충분히 잠을 못 이루고 일찍 깨거나 밤 사이 자주 깨는 증상을 보인다. 많은 환자가 식욕감소와 체중저하를 보이는데 일부 환자는 식욕이 증가하고 수면이 길어지는 비전형적 양상을 보이기도 한다. 또한 집중력 저하와 같은 인지기능 저하 증상도 상당수에서 나타날 수 있다.

2.1.2. 우울증 진단 방법

정신상태 검사로 우울증이 의심되면 우선 우울증을 일으킬 수 있는 다른 질환에 대한 감별 진단이 우선시 되어야 한다. 갑상선 기능 저하증 등의 내분비 질환과 같은 내과 질환에 대한 감별이 필요하고 뇌졸중과 같은 신경과적 문제에서도 우울증이 발생할 수 있다. 다양한 질환이 우울증과 연관성이 있으므로 환자 증상에 따른 정밀검사가 필수적이다. 또한 우울 증상은 다른 정신과 질환의 증상 중 하나로 나타날 수 있으므로 불안 장애, 양극성 장애 등과의 감별이 필요하나 두 질환 이상이 공존하는 경우도 흔하다.

우울증은 정신건강의학과 의사와 면담하여 병력을 청취하고 환자의 상태가 진단 기준에 부합하는지 확인한 후 진단한다. 이 과정에서 심리 검사를 통해 다른 정신 질환의 공존 여부, 지능 등 필요한 정보를 보충할 수 있다. 다른 신체 질환에 의한 이차적 우울증을

감별하기 위해 혈액학적 검사 및 뇌영상 검사 등을 시행할 수 있다. 특히 우울증의 진단 기준인 **DSM-IV**이 있는데, 이것은 다음과 같다.

우울증 진단 기준(DSM-IV)

- ① 2주 이상, 거의 매일 지속되는 우울한 기분
- ② 일상 대부분의 일에서 관심 및 흥미 감소
- ③ 식욕 감소 또는 증가(체중 감소 또는 증가, 한 달에 5% 초과)
- ④ 불면 또는 과다 수면
- ⑤ 정신운동 지연 또는 정신운동 초조
- ⑥ 피곤 또는 에너지의 감소
- ⑦ 무가치감, 부적절한 죄책감
- ⑧ 집중력 저하, 우유부단
- ⑨ 반복적인 자살 생각

위에서 언급한 증상 중 5개 이상(1, 2번 중에 하나 이상 포함)이 있고, 이러한 증상이 일상생활을 심각하게 저해하면 우울증으로 진단할 수 있다.

2.1.3. 우울증의 원인



<서울 아산병원 우울증의 원인>

우울증의 확실한 원인은 아직 명확하게 밝혀지지 않았지만, 다만 다른 정신 질환과 마찬가지로 다양한 생화학적, 유전적, 환경적 요인이 우울증을 일으킨다.

- 생화학적 요인

뇌 안에 있는 노르에피네프린, 세로토닌, **GABA**등과 같은 신경 전달 물질과 갑상선, 성장 호르몬, 시상하부-뇌하수체-부신피질 축과 같은 호르몬 이상, 생체 리듬의 변화와 관련이 있다.

- 유전적 요인

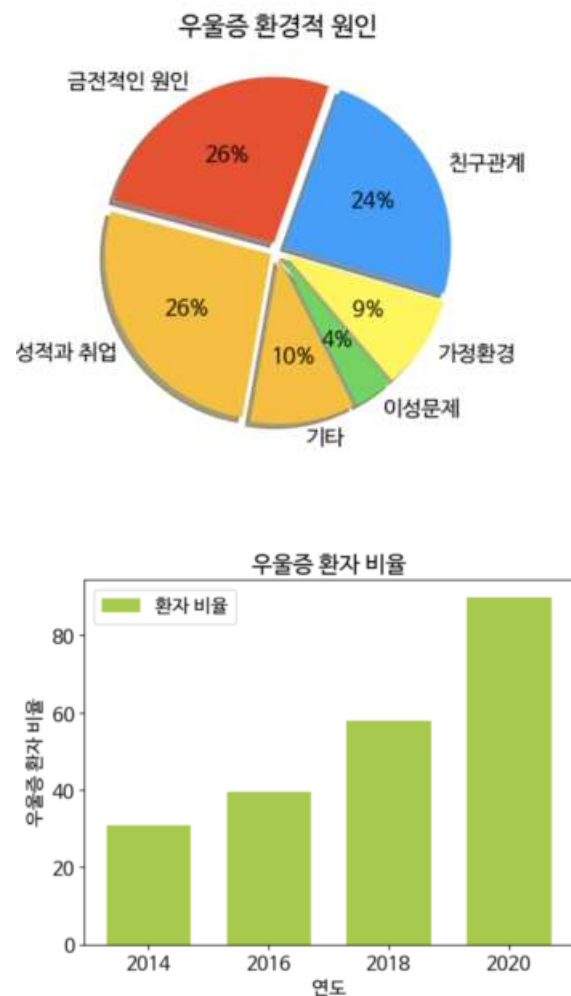
일란성 쌍둥이의 경우, 한 명이 주요 우울증을 앓고 있으면 다른 한 명도 우울증이 걸릴 확률이 **50%** 정도가 된다. 이것을 통해, 주요 우울증 발병에 유전적 요소가 작용하는 것은 명확하다는 것을 알 수 있다. 그러나 유전적 요소로 설명되지 않는 요인도 우울증 발병에 많은 영향을 끼친다.

- 환경적 요인

스트레스만으로 우울증이 생기는 것은 아니지만, 스트레스는 우울증 증상 발현에 영향을 준다고 한다. 살아가면서 대처하기 어려운 상황이 우울증 유발의 환경적 요인이 된다. 사랑하는 사람을 잃는 것, 경제적 문제, 강한 스트레스 등을 예로 들 수 있다.

2.1.4. 우울증 환자 증가 배경

최근 들어 우울증이 급증하는 이유는 물론 생화학적 요인이나 유전적인 요인도 있겠지만 그것보다는 환경적인 요인으로 인한 스트레스가 주요원인으로 나타났다. 그리고 우울증에 대한 이유는 성적과 취업, 금전적인 원인, 친구관계, 가정환경, 이성문제와 기타등으로 나타났다.



<20만명을 기준으로 퍼센트로 변환하여 데이터 시각화 한 것>

이를 통해 우리는 점점 복잡해지는 사회 환경과 이에 따른 학업 및 취업 문제, 금전적인 원인이나 다른 많은 기타 이유로 우울증 환자가 매년 증가하고 있음을 확인 할 수 있다.

2.1.5. 우울증 치료 방법 및 예방 방법

우울증의 치료 방법에는 크게 약물 치료와 정신 치료, 전기 경련 요법, 반복적 경두개 자기자극법 등이 있다.

- 약물 치료

현재 사용되는 항우울제는 대부분 비슷한 효능을 보인다. 약물 투여 2~3주 후에 효과를 보이기 시작하며, 대개 4~6주 정도 지나면 충분한 효과가 나타난다. 약물을 충분한 용량으로 충분한 기간 동안 사용하면 전체 환자의 2/3에서 효과가 나타난다. 약물 치료로 우울증 증상이 호전되더라도, 6개월 정도는 약물 치료를 계속해야 재발을 예방할 수 있다. 이렇게 약물을 장기간 사용해도 신체에 특별한 위험성은 없다. 중독성도 거의 없다.

- 정신 치료(심리 요법)

우울증을 유발한 스트레스에 대처하는 능력을 향상시킴으로써 현재의 증상을 조절하는 치료 방법이다. 이뿐만 아니라 우울증이 재발하지 않도록 개입하는 효과가 있다. 정신 치료를 효과적으로 받으면 우울증이 치료될 뿐만 아니라 전반적인 정신 건강도 향상되어서, 치료 전보다 만족스러운 삶을 살게 되기도 한다.

- 전기 경련 요법(Electroconvulsive therapy)

우울증의 가장 효과적인 치료법 중 하나이다. 다른 치료에 비해 효과가 아주 빨라서 수일 내지 1~2주면 치료 효과를 기대할 수 있다. 합병증이 발생할 확률도 적다. 자살 위험성이 높은 경우, 신체 쇠약이 심해서 빠른 치료가 필요한 경우, 항우울제에 반응이 없는 경우에 사용한다.

- 반복적 경두개 자기자극법(Repetitive transcranial magnetic stimulation; rTMS)

경두개 자기자극으로 뇌가 자극되면 일련의 복잡한 과정을 거쳐 흥분성 또는 억제성 효과가 나타난다. 경두개 자기자극의 기술적 원리는 코일에 짧고 강한 전류를 흘려 급격히 변화하는 자기장을 유발하고, 이 자기장이 다시 전기장을 유발해 사람의 두피 위를 자극할 때 대뇌의 신경세포를 흥분시키는 것이다. 다양한 정신-신경계 질환이 있는 환자에게 치료용 목적으로 사용한다. 우울증에 효과가 있다고 알려져 있다.

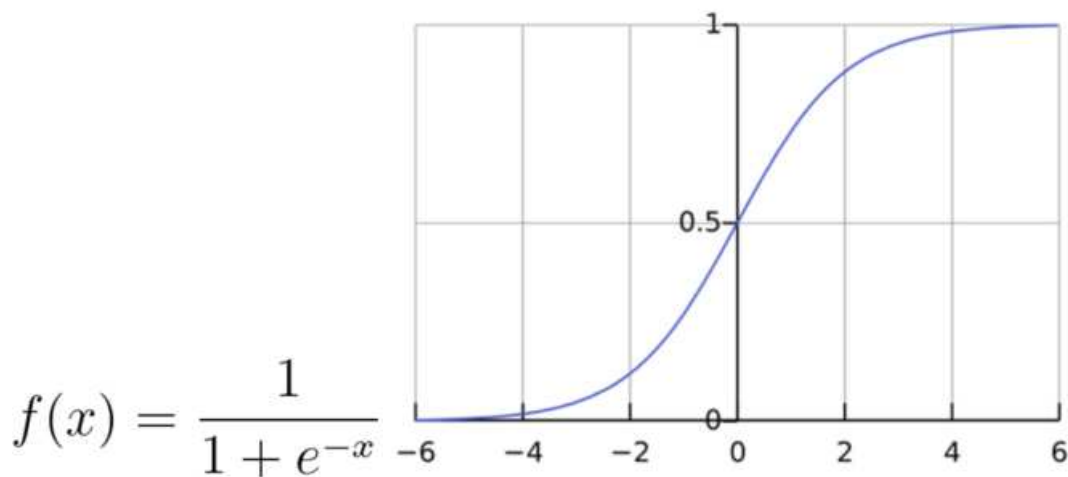
- 우울증 예방 방법

1. 규칙적인 운동이 도움이 된다.
2. 햇볕을 쬐어 주는 것이 도움이 된다.
3. 규칙적이고 균형 잡힌 식사를 한다.
4. 술과 커피, 담배를 줄인다.

5. 공연한 부정적인 생각을 버린다.
6. 자신만의 취미를 가져라

2.1.6. 로지스틱 회귀

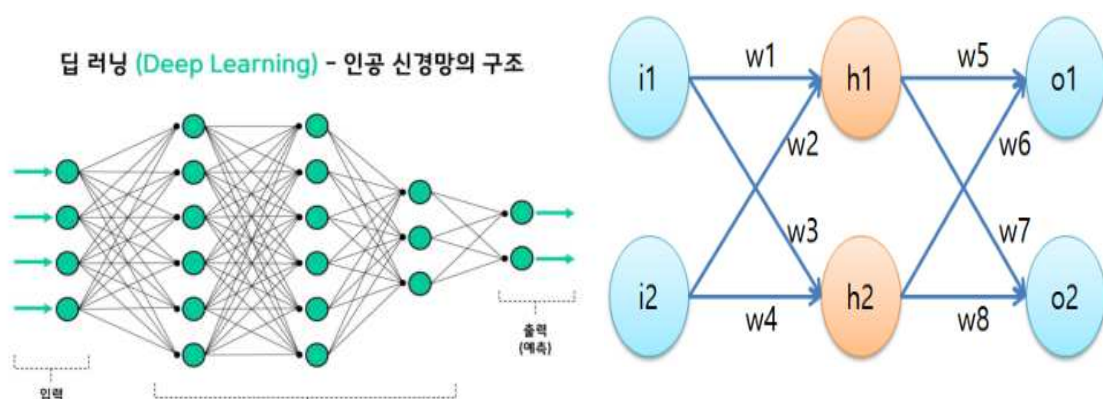
로지스틱 회귀란, 회귀를 사용하여 데이터가 어떤 범주에 속할 확률을 0에서 1사이 값으로 예측하고, 그 확률에 따라 가능성이 높은 범주에 속하는 것으로 분류해주는 지도 학습 알고리즘을 말한다. 로지스틱 회귀는 **Log Odds**를 이용한다. **Odds**란, 결과가 1일 확률/결과가 0일 확률으로 0부터 무한대까지의 값을 얻을 수 있다. 만약 우울증에 걸릴 확률이 0.7이면, $Odds = 0.7/0.3 = 2.333...$ 이다. 그리고 이 값에 $\log$ 를 취한 것이 **Log Odds**이고, 앞의 예시에 적용해보면, $\log(2.333...) = 0.847$ 이 나온다. **Log Odds**함수는 $-\infty$ 부터 ∞ 까지의 값을 얻을 수 있다. 따라서, 결과가 0이나 1로 나와야 하기 때문에, **Log Odds**를 통해 나온 값을 0과 1사이의 값으로 변환해주기 위해서 **Sigmoid** 함수를 사용한다. **Sigmoid** 함수는 다음과 같다.



Sigmoid까지 적용하면 사건이 발생할 확률이 나오게 된다. 그 확률이 0.5보다 크면 1, 0.5보다 작으면 0으로 분류되는 것이다.

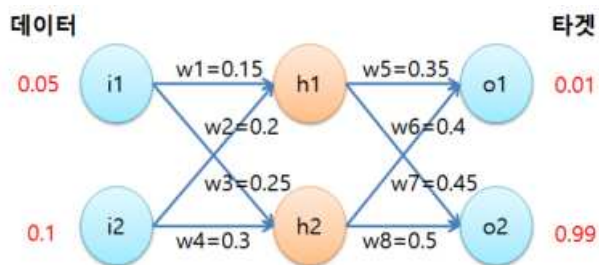
Sigmoid함수 이외에도 다중 분류류에 사용되는 **softmax**와 같은 다른 함수를 사용하기도 한다.

2.1.7. 딥러닝



딥러닝은 다층구조 형태의 인공 신경망을 기반으로 하는 머신러닝의 한 종류로, 다량의 데이터로부터 높은 수준의 추상화 모델을 구축할때 쓰는 기법이다. 이때, 컴퓨터가 사람처럼 스스로 학습할 수 있게 하기위해 인공 신경망을 구축해 학습하는데, 인공 신경망은 데이터 입력, 가중치 및 편중의 조합을 통해 인간의 뇌 속 신경망과 유사하게 인공적인 신경망을 만들어서 인간의 직감과 비슷하게 작용하도록 만든 기계학습 알고리즘이다.

- 가중치 : 딥러닝에서 다음 층으로 넘어가는 사이 사이에 값에 얼마나 곱해야 할지를 결정하는 것으로, 결과에 그 값이 얼마나 영향을 줄지 조절하는 매개변수이다.
- 편향 : 하나의 뉴런으로 입력된 모든 값을 다 더한 다음에 이 값에 더 해주는 상수이다



이 데이터에 입력된 데이터의 샘플이 (0.05, 0.1)이고 타겟값이 (0.01, 0.99)라고 할 때, 은닉층은 가중치와 입력값의 곱에 편향을 더한 값을 받는다. 예를 들어, net_{h1} 은 $i1*w1 + i2*w2 = 0.5*0.15 + 0.1*0.2 = 0.0275$ 이다. 이 값이 들어오면, 0과

1사이의 값을 출력해주는 sigmoid 함수를 이용해 out를 구한다. net_{h1} 에 적용시키면, out_{h1} 은 0.5069라는 값이 나온다. 동일한 방식으로 구한 out_{h2} 의 값은 0.6186이다. 동일한 방법으로 다음 층의 값을 구하면, $o1$ 은 $net_{o1} = 0.3817$, $out_{o1} = 0.5943$, $o2$ 는 $net_{o2} = 0.4834$, $out_{o2} =$

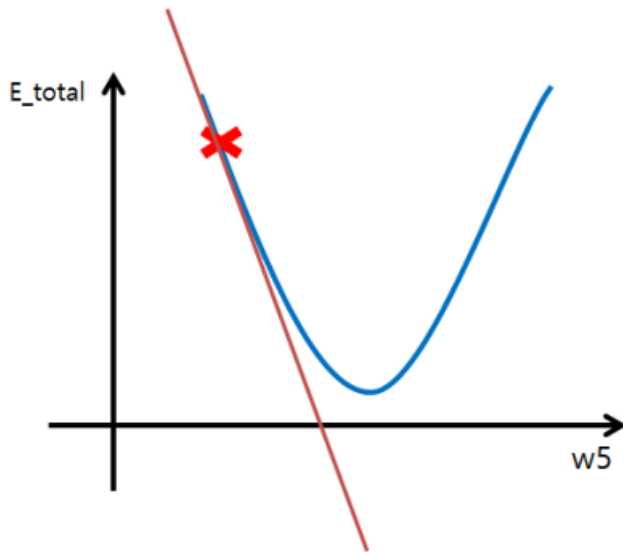
0.6186이 나온다.

그리고 이 값을 원래의 타겟값과의 오차를 구해야한다. 오차는 오차제곱의 합, 즉

$$E_{total} = E_{o1} + E_{o2}$$

$$= \frac{1}{2}(target_{o1} - out_{o1})^2 + \frac{1}{2}(target_{o2} - out_{o2})^2$$

와 같은 식을 이용해 구한다. 따라서 out_{o1} 과 out_{o2} 의 오차는 0.2397이다. 이 오차의 최소값이 가중치로 편미분 했을 때 0이 되는 지점이다.



<기존의 8차원 그래프를 2차원의 그래프로 나타낸 것이다.>

편미분 했을 때의 값, 즉 기울기가 음수인지 양수인지에 따라 기울기를 0으로 만들기 위해 가중치를 옮겨줘야한다. 그리고 그 이동하는 정도를 학습률(learning rate)라고 한다.

가중치를 갱신할 때 새로운 가중치는 $wn^+ = wn - a \frac{\partial E_{total}}{\partial wn}$ 와 같은 식으로 구한다.

(여기서 a 는 학습률이다.) 위의 식을 통해 가중치들을 갱신하준 후, 다시 E_{total} 을 확인하는 과정을 여러번 반복하면서 오차를 점점 줄여나가며 모델을 학습시킨다.

2.1.8. AI 모델에서 사용된 파라미터

sklearn - 기계학습 모델에서 사용되는 알고리즘이고, **sklearn** 즉 **scikit - learn**의 알고리즘 차트의 시트이다. **sklearn**을 이용하여 기계학습을 실행할 때, 우리의 분석하는 것에 대해 적절한 모델을 선택하는 것을 도와준다. **sklearn**의 분석에는 분류, 회귀, 클러스터링 등이 있는데, 우리는 **sklearn**에서 회귀중에서도 **Linear SVR**(선형회귀)을 하는데에 사용했다.

- **Ridge, Liner SVR** : 중소규모(10만건 미만)으로 설명 변수의 전부가 중요한 경우에 사용하는 회귀 분석 방법이다.
- **SVR(가우스 커널), Ensemble : Ridge 또는 Liner SVR**로 잘 분석되지 않는 경우 사용하며, 비선형적 회귀 분석 방법이다.

3. 클러스터링 - 전달된 데이터를 어떤 규칙에 따라 나누어준다. 선생이 없는 학습 클러스터링 문제를 해결해주는 알고리즘이다.

- **KMeans** : 몇 개의 클러스터를 분할할지, 사전에 정해둔 경우 추천하는 클러스터링 분석 방법이고, 대규모 데이터는 **MiniBatch**를 사용한다.
 - **MiniBatch** : 데이터를 나누면서 학습시키는 방법
- **스펙트럼 클러스터링(GMM)** : **KMeans**로 잘 분석되지 않는 경우에 이용하며, 비선형적인 클러스터링 분석 방법이다.
- **MeanShift, VBGMM** : 몇 개의 클러스터로 나눌지에 대해, 사전에 정할 수 없는 경우에 사용하는 클러스터링 분석 방법이다.

4. 기타 기능

- **차원감소** : 전달된 데이터의 차원 수가 많은 경우, 학습효율이 떨어지므로 차원을 감소시키는 전처리를 시행해줌
 - **PCA, 커널PCA, Isomap, SpectralEmbedding**
- **하이퍼 파라미터의 최적화** : 기계학습을 실행할 때, 학습 방법 등을 조정하는 수치를 하이퍼 파라미터라고 하는데, 이것을 계산하기 편하게 최적화시켜줌.
 - **그리드 서치, 크로스밸리테이션**

Adam(Adaptive Moment Estimation) : 딥러닝 최적화 기법 중 하나로서 **Momentum**과 **RMSProp**의 장점을 결합해서 만든 알고리즘이다. 즉, 학습의 방향과 크기를 모두 개선한 기법으로 딥러닝에서 가장 많이 사용되는 최적화 기법이다.

- **momentum**의 한계점 : **Local Minimum**에 빠지기 쉬움, 안장점(**Saddle point**)을 벗어나지 못함.
- **RMSProp**의 장점 : 변수마다 적절한 학습률을 적용하여 효율적인 학습 가능, 보다 학습을 오래할 수 있음.

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \nabla f(x_{t-1})$$

β_1 : Momentum의 지수이동평균 ≈ 0.9

$$g_t = \beta_2 g_{t-1} + (1 - \beta_2) (\nabla f(x_{t-1}))^2$$

β_2 : RMSProp의 지수이동평균 ≈ 0.999

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \hat{g}_t = \frac{g_t}{1 - \beta_2^t}$$

$\hat{m}, \hat{g}$: 학습 초기 시 m_t, g_t 가 0이 되는 것을 방지하기 위한 보정 값

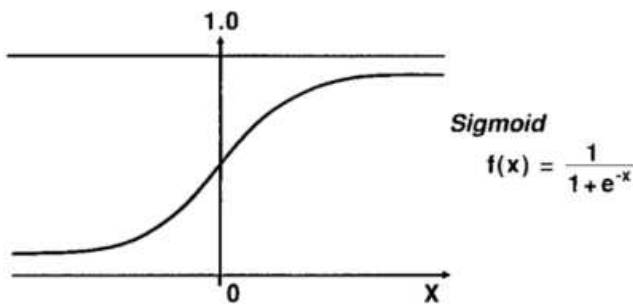
$$x_t = x_{t-1} - \frac{\eta}{\sqrt{\hat{g}_t + \epsilon}} \cdot \hat{m}_t$$

ϵ : 분모가 0이 되는 것을 방지하기 위한 작은 값 $\approx 10^{-8}$

η : 학습률 ≈ 0.001

1. 지수이동평균 : m_t 와 g_t 는 각각 Momentum과 RMSProp에서 사용한 수식과 동일한데, β 는 지수이동평균으로써 하이퍼파라미터이다. β_1 은 0.9가 가장 좋은 값이고, β_2 는 0.999가 가장 좋은 값이다. 지수 이동평균은 예전 time step에서의 값은 적게 반영하고 최근 step의 값을 많이 반영하기 위해 해준다..
2. 편향 보정 : m_t, g_t 의 초기값이 0에 가까워지는 편향(bias)의 문제를 해결하기 위해 $\hat{m}, \hat{g}$ 을 만드는 과정을 편향 보정(bias correction)이라고 한다. 학습이 계속 진행되다 보면 $(1-\beta_1)$ 와 $(1-\beta_2)$ 는 거의 1에 가까워지기 때문에 $\hat{m}, \hat{g}$ 는 결국 m_t, g_t 와 같은 값이 된다.
3. 학습률 : 학습률은 어떠 비율만큼 파라미터를 업데이트할지 결정하는 값으로써 0과 1 사이의 값에서 나오는 하이퍼파라미터이다.
 - Adam 알고리즘에서 학습률은 0.001이 가장 좋은 설정 값이다.

sigmoid - 시그모이드는 활성화 함수의 한 종류로, 왼쪽의 그래프와 같이, 0과 1사이의



실수를 반환해주는 함수이다.

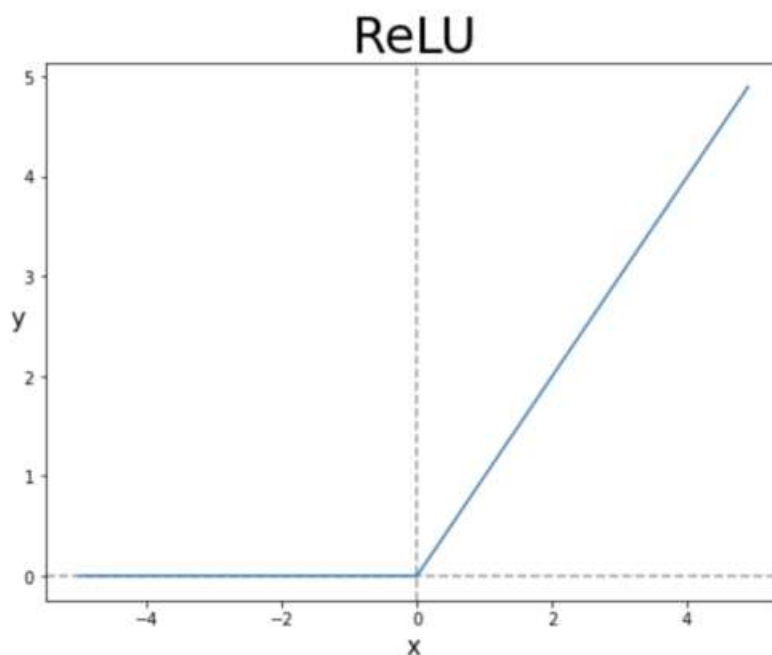
시그모이드 함수의 장점은, 출력 값의 범위가 0과 1 사이의 값이기 때문에 가중치를 갱신하는 과정에서 기울기 크면, 가중치가 너무 커져 발산하게 되는 오류인 기울기 폭주(Gradient Exploding)가

발생하지 않는다. 또한, 평범한 범주를 벗어난 값인 이상치가 들어온다고 하더라도 0이나 1에 수렴하기 때문에 이 문제도 해결된다. 하지만, 시그모이드 함수의 단점은 기울기 폭주와 반대되는 기울기 소실(Gradient Vanishing)이 발생할 수 있다는 것이다. 시그모이드 함수는 입력되는 값이 아무리 크더라도 0과 1사이의 좁은 범위의 값이 출력되기 때문에 곱할수록 점점 기울기가 작아지게 된다. 결국 기울기가 0에 수렴하면서 가중치가 거의 변하지 않게 되는 것이다.

따라서 시그모이드는 은닉층에서 시그모이드 함수를 사용하기에는 적합하지 않으며, 이진 분류를 해야하는 출력층에서만 사용하는 것이 적합하다.

relu - 은닉층에서 굉장히 많이 사용되는 활성화 함수이다. 은닉층에 어떤 활성화 함수를 써야할지 모르겠다 싶으면 **relu** 함수를 그냥 써도 될 정도로 은닉층에서 자주 사용되는 활성화 함수이다. 이 **relu** 함수는 정류된 선형 함수로 이루어져있는데, 이걸 **+, -**가 반복되는 신호에서 - 흐름을 차단하여 음수일때는 **0**을 반환해주고, **+**가 나오면 그 값 자기 자신을 반환해준다. 동시에 **relu** 함수의 공식은 음수면 **0**, 양수면 자기 자신을 반환해주는 간단한 공식이기 때문에 경사하강법을 할 시 다른 활성화 함수에 비해 학습 속도가 매우 빠르다. 또한, 이전에 **Sigmoid** 함수에서 있던 기울기 소실 문제를 해결하였다.

- 기울기 소실 문제가 발생하지 않는 이유 : **relu** 함수는 양수는 그대로 음수는 **0**으로 반환하기 때문에 특정 양수 값에 수렴하지 않아, 출력값의 범위가 넓고 양수인 경우 자기 자신을 그대로 반환해주기 때문에, 심층 신경망인 딥러닝에서 **sigmoid** 함수를 활성화 함수로 사용할 때 발생했던 문제인 기울기 소실 문제를 해결할 수 있다.



한계점 : 음수 값이 들어오는 경우 모두 **0**으로 반환하기 때문에 입력값이 음수인 경우 기울기도 전부 **0**으로 나오게 되고, 결국 가중치 업데이트가 안되는 현상이 발생하게 된다.

- **Binary classification**(이진분류) : 두 그룹중 하나로 분류하는 일반적인 머신러닝 기법이다.
- **binary cross entropy** : 출력층에서 **sigmoid** 함수를 사용하는 이진분류(**Binary Classification**)에서 이항 교차 엔트로피라고도 부르는 손실 함수이다.

비용함수 : 학습데이터에 존재하는 전체의 에러 정도를 의미함. 즉 오차, 모델이 설명하지 못하는 정도를 나타낸다. 그래서 이 파라미터의 최적 값은 비용함수의 값을 최소화하는 값이라고 볼 수 있다.

- 대표적인 비용함수에는 MAE, MSE, RMSE가 있다.

- $\frac{1}{N} \sum_i |y_i - \hat{y}_i|$ (mean absolute errors)
- $\frac{1}{N} \sum_i (y_i - \hat{y}_i)^2$ (mean squared errors)



정규방정식(Normal Equation) : 비용함수가 아래로 볼록한 2차 방정식 형태(convex form)인 경우, 미분해서 기울기가 0인 지점을 계산하면 비용함수를 최소화하는 지점을 찾을 수 있다. 하지만 대부분 실제 딥러닝의 비용함수는 위와 같은 형태로 복잡하게 나타나기에 이것을 사용하는 것은 어렵다.

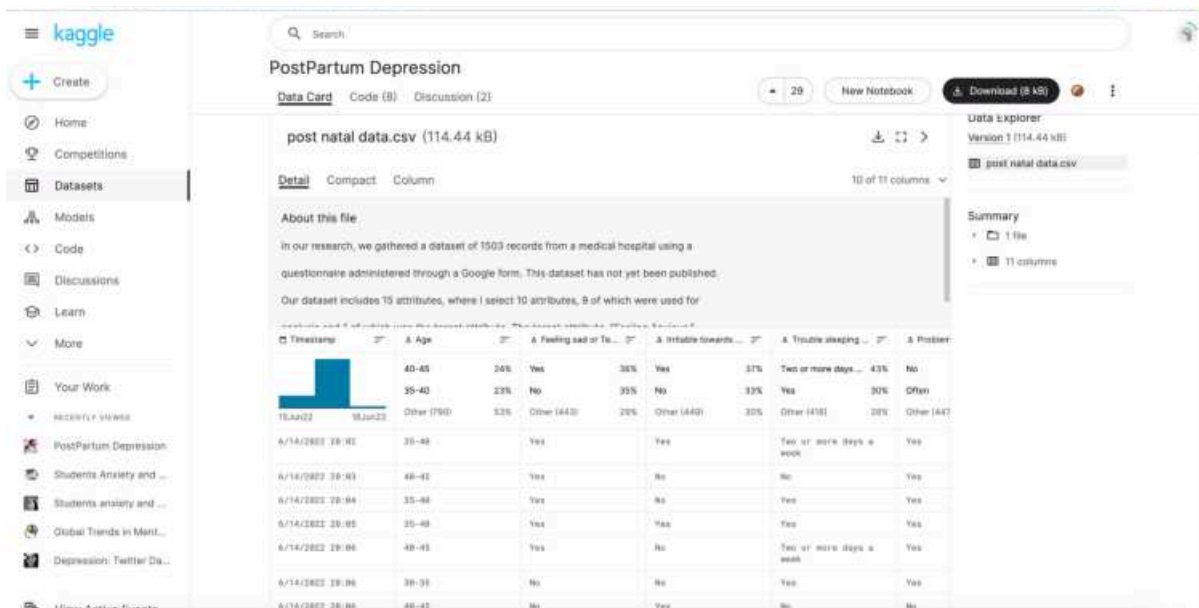
경사하강법(Gradient Descent) : 앞서 얘기했던 것과 같이 대부분이 위와 같이 복잡한 형태로 나오기 때문에 딥러닝 비용함수에서는 Normal Equation을 사용할 수 없기 때문에 경사하강법을 사용해주다. 경사하강법은 한번에 최적의 비용함수 감소 지점을 찾을 수 없기 때문에, 임의의 지점에서 시작해서 값을 업데이트해 가면서 비용함수를 최소화하는 지점을 탐색하는 방법을 얘기한다.

하이퍼 파라미터 (Hyper Parameter) : 컴퓨터가 아니라 사람이 지정하는 파라미터를 말한다.

- 학습률(Learning Rate)

2.2. 연구 진행

2.2.1. 데이터 수집

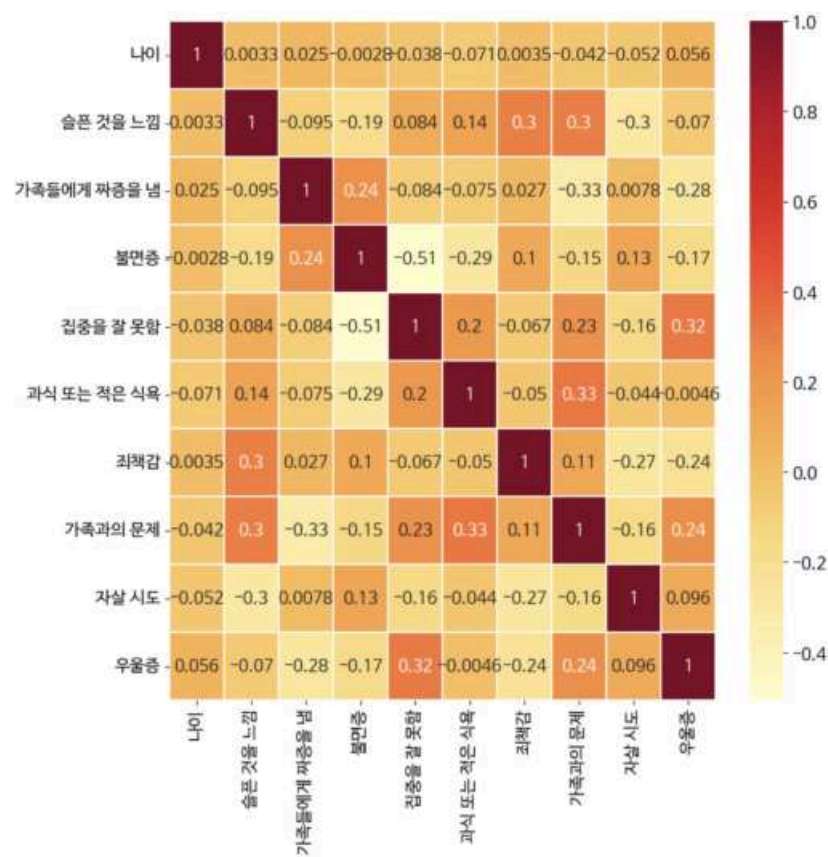


먼저, 우울증 환자들의 데이터를 이용하여 인공지능을 학습시켜야 한다. 그러나 이것이 의료와 관련된 데이터였었기 때문에 데이터를 수집하는 것부터 생각보다 많은 제한이 있었고, 인공지능 학습을 시키기 위해서는 우울증 환자의 데이터중 아무거나 사용을 하면 안되고, 어느 정도 일정한 형식에 맞춰진 데이터를 사용해야하기 때문에 이러한 데이터를 국내 사이트에서는 찾을 수 없어 사용할 수 없었다. 그래서 우리는 학습시키기 위한 데이터 세트들을 제공하는 캐글(kaggle) 혹은 natal등의 해외 사이트에서 1503명의 우울증 환자 데이터를 수집할 수 있었다. 우리가 수집한 환자 데이터에는 환자의 나이, 가끔 이유없는 슬픔을 느낄때가 있는지, 가족들에게 짜증을 내는지, 잠을 자는데 문제가 있는지, 집중을 잘 못하는지, 과식이나 적은 식욕을 가지고 있는지, 혹은 죄책감을 가지고 있는지, 가족과의 문제나 자살 시도와 같은 총 9개의 속성과 우울증의 양성, 음성 여부를 나타내는 각각 하나의 클래스가 포함되어 있다. 나는 이것을 컴퓨터가 이해할 수 있게 Yes를 1, No를 0, Sometimes는 2로 변환시켜주었다.

2.2.2. 우울증과 속성들 간의 상관관계

우울증의 양성, 음성을 나타내는 클래스와 우울증과 관계가 있는 환자의 나이, 가끔 이유없는 슬픔을 느낄때가 있는지, 가족들에게 짜증을 내는지, 잠을 자는데 문제가 있는지, 집중을 잘 못하는지, 과식이나 적은 식욕을 가지고 있는지, 혹은 죄책감을 가지고 있는지, 가족과의 문제나 자살 시도를 포함한 9가지 속성들의 상관관계를 알아보기 위하여 seaborn 라이브러리를 이용한다. seaborn을 사용하여 한 개의 클래스와 9개의 속성들을 데이터 분포 시각화 방식들 중 하나인 히트맵을 통해 나타내었다. 히트맵에서는 겹치는 부분의 숫자가 상대적으로 크거나 어둡고 진한 붉은색으로 표시되면 이것의 상관관계가 높다는 것을

의미하고, 숫자가 0이하이거나 색이 연한 노란색으로 표시되면 반비례하는 상관관계를 가진다는 뜻이다. 그와 반대로 겹치는 부분의 숫자가 0에 가까워지거나 연한 주황색으로 표시되면, 이것의 상관관계가 낮다는 것을 의미한다.



클래스와 속성들 간의 상관관계를 나타낸 히트맵이 10x10으로 구성된 그래프에서 우울증과 가장 상관관계가 있는 요인은 집중을 잘 못하는 것으로 나타났는데, 고려대 안산병원 정신건강의학과 한창수 교수가 조사한 바에 따르면, 병원을 방문한 환자들을 대상으로 신경심리검사를 실시한 결과 경도인지장애를 가진 환자들 중 우울증상을 동반한 환자들이 기억력과 집중력이 일반인에 비해 떨어진다는 것을 알아냈다. 따라서 집중을 잘 못하는 것이 우울증과 큰 관련이 있다. 이에 잇따라서 2번째로 우울증과 연관이 있던 요인은 가족과의 문제인데, 이것은 앞에서 얘기했었던 서울아산병원의 자료중에, 환경적 요인들중 가정 환경의 문제나 이혼이나 사별, 부모님과의 성적이나 취업으로 인한 싸움으로 설명할 수 있다. 그다음으로 3번째로 우울증과 연관이 있던 요인은 자살 시도였는데, 우울증이 심화되면 발생하는 것이 자살 시도이기도 하고, 이것이 우울증의 증상중 하나이기 때문에 이것이 우울증과 연관이 있다고 말 할 수 있다. 우리는 우울증뿐만 아니라 각각의 속성들 사이에도 연관성이 있는 것을 확인할 수 있다. 자살 시도가 우울증의 증상중 하나인 불면증과 큰 연관성이 있다는 것 뿐만 아니라, 슬픈 것을 느끼는 게 가족과의 문제가 있다는 것과 연관이 있는 등 우리는 이러한 히트맵을 통해 다양한 것들을 알 수 있다.

2.2.3 우울증예측 로지스틱 회귀, 딥러닝 모델의 제작 과정 & 비교

이 모델의 개발 환경은 수정이 편리하고 자동저장 기능이 있는 구글 colab에서 이 프로그램을 만들게 되었다. 그리고 인공지능과 데이터 분석에 편리한 **pandas, numpy, tensorflow, keras, linear**와 같은 라이브러리들을 활용하기 위해 파이썬을 개발을 위한 프로그래밍 언어로 사용하였다. 또한 로지스틱 회귀와 딥러닝 모델 두개를 우리가 사전에 얻은 데이터를 통해 제작한 후, 두개의 정확성을 비교하여 우리의 데이터에 더 적합한 모델을 골라서 이 모델을 사용할 것이다.

로지스틱 회귀, 딥러닝 모델의 제작과정

1. 라이브러리 불러오기	구글 Colab에 들어가서 머신러닝과 딥러닝, 데이터 처리에 필요한 matplotlib, numpy, tensorflow, pandas, keras, sklearn, linear 등의 라이브러리를 불러온다.
2. 데이터 불러오기 및 데이터셋 분리	#데이터 및 레이블 분리하기 lim = int(len(df)*0.7) #7:3 지점 저장 train_x = df.iloc[:lim, 0:9].to_numpy() test_x = df.iloc[lim:, 0:9].to_numpy() train_y = df.iloc[:lim, 9].to_numpy() test_y = df.iloc[lim:, 9].to_numpy() 우리가 수집한 데이터를 URL을 통해 csv 파일 형식으로 읽어온 뒤, 총 1503명의 우울증 환자 데이터를 훈련 데이터인 train을 70%, 테스트 데이터인 test 30%로 분리한다. 7:3으로 분리하는 것이 가장 높은 정확도를 보임 <pre>graph TD; Dataset[Dataset] --> Training1[Training]; Dataset --> Testing1[Testing]; Training1 --> Training2[Training]; Training1 --> Validation[Validation]; Training1 --> Testing2[Testing];</pre>
3. 로지스틱 회귀 모델	LogisticRegression 모델로 로지스틱 회귀 모델을

구성	구성한다. <pre>#이항분류 모델구성하기 from sklearn.linear_model import LogisticRegression log_reg = LogisticRegression(solver='lbfgs')</pre> <pre>#로지스틱 회귀 학습하기 model = log_reg.fit(train_x, train_y)</pre> 로지스틱 회귀는 데이터 묶음을 분류하기 위해 방정식을 계산하게 되는데 이 방정식은 가장 알맞은 매개변수의 집합을 찾는 것이다. 따라서 우리는 파이썬 머신러닝 라이브러리 SKLEARN을 불러와 로지스틱 회귀 모델을 만들어 학습한 후, 실제 환자의 우울증 여부를 이진 분류를 통해 0과 1로 나타내었다.
4. 로지스틱 회귀 학습	<pre>#로지스틱 회귀 학습결과 w = model.coef_ b = model.intercept_ print(w) print(b)</pre> <pre>[[0.16365194 -0.21355205 -0.54389294 0.05829714 0.82572469 -0.34643386 -0.58054993 0.5659512 0.38276982]] [-0.67472265]</pre> 앞에서 분리했는 데이터를 모델에 학습을 시킨 후, 가중치 w와 편향 b를 확인한 후 모델 테스트를 시킨다.
5. 학습 결과 및 정확도 확인	<pre>#모델 테스트 Y_prediction = model.predict(test_x).flatten() for i in range(15): label = test_y[i] prediction = Y_prediction[i] print("실제 우울증 여부: {:.0f}, 학습후 예측 우울증 여부: {:.0f}".format(label, prediction))</pre> <pre>실제 우울증 여부: 1, 학습후 예측 우울증 여부: 1 실제 우울증 여부: 1, 학습후 예측 우울증 여부: 1 실제 우울증 여부: 1, 학습후 예측 우울증 여부: 1 실제 우울증 여부: 0, 학습후 예측 우울증 여부: 0 실제 우울증 여부: 1, 학습후 예측 우울증 여부: 1 실제 우울증 여부: 0, 학습후 예측 우울증 여부: 0 실제 우울증 여부: 1, 학습후 예측 우울증 여부: 1 실제 우울증 여부: 1, 학습후 예측 우울증 여부: 1 실제 우울증 여부: 1, 학습후 예측 우울증 여부: 1 실제 우울증 여부: 1, 학습후 예측 우울증 여부: 1 실제 우울증 여부: 1, 학습후 예측 우울증 여부: 1 실제 우울증 여부: 1, 학습후 예측 우울증 여부: 1 실제 우울증 여부: 1, 학습후 예측 우울증 여부: 1 실제 우울증 여부: 0, 학습후 예측 우울증 여부: 0 실제 우울증 여부: 1, 학습후 예측 우울증 여부: 1 실제 우울증 여부: 0, 학습후 예측 우울증 여부: 0</pre> <pre>#모델 평가하기 y_pred = model.predict(test_x) print("테스트 세트의 정확도 : {:.2f}".format(model.score(test_x, test_y)))</pre> 테스트 세트의 정확도 : 0.86 분리한 데이터로 로지스틱 회귀 모델을 테스트 해본 결과, 86%라는 정확도가 나왔다.

6. 딥러닝 모델 구성

#딥러닝 모델 구성하기

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

model = Sequential()
model.add(Dense(30, input_dim=9, activation='relu'))
model.add(Dense(1, activation='sigmoid'))
```

Sequential 모델로 딥러닝 모델을 구성했으며, 은닉층에 뉴런의 수를 30개로 지정했고, 활성화 함수로 ReLU를 사용했다. 또한, 출력층에는 Sigmoid를 활성화 함수로 사용하였다.

7. 딥러닝 모델 컴파일

#모델 컴파일

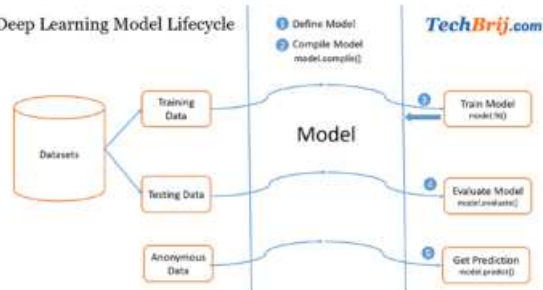
```
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
#loss='mse'
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 30)	300
dense_1 (Dense)	(None, 1)	31

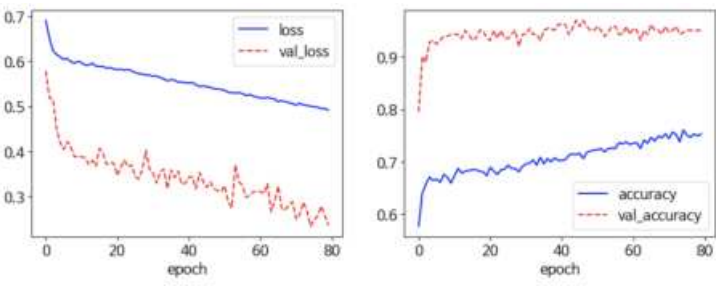
Total params: 331
Trainable params: 331
Non-trainable params: 0

Deep Learning Model Lifecycle

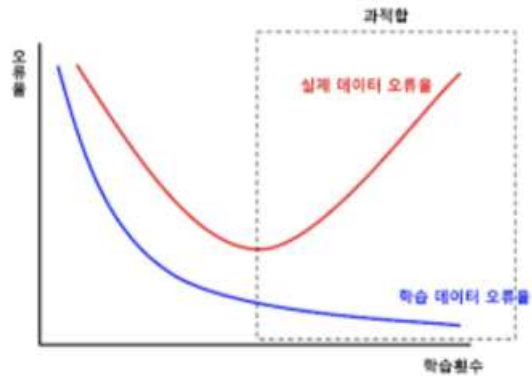


손실

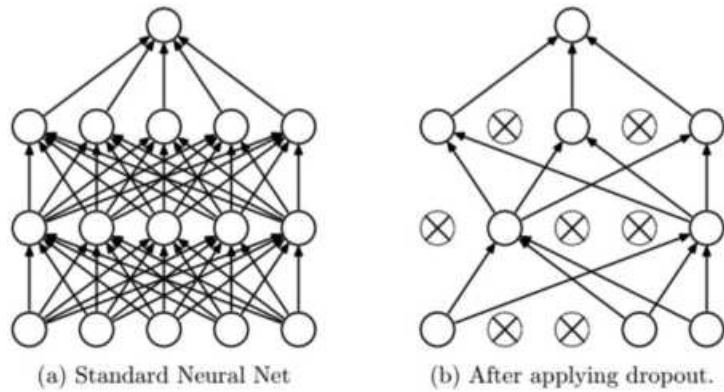
함수에는 이진 분류에 사용되는 `binary_crossentropy`를 적용했으며, `optimizer` 즉, 최적화 알고리즘은 `adam`을 사용하였다. 비용함수는 경사하강법(Gradient Descent)을 사용하였고, `metrics`에는 정확히 예측한 비율을 나타내는 `accuracy`를 이용했다.

8. 딥러닝 모델 학습	<pre>#모델 학습 history = model.fit(train_x, train_y, epochs=80, batch_size=10, validation_split=0.25)</pre> 79/79 [=====] - 6s 3ms/step - loss: 0.5358 - accuracy: 0.7224 - val_loss: 0.3244 - val_accuracy: 0.9582 Epoch 52/80 79/79 [=====] - 6s 4ms/step - loss: 0.5318 - accuracy: 0.7258 - val_loss: 0.2881 - val_accuracy: 0.9506 Epoch 53/80 79/79 [=====] - 6s 4ms/step - loss: 0.5303 - accuracy: 0.7186 - val_loss: 0.2761 - val_accuracy: 0.9392 Epoch 54/80 79/79 [=====] - 6s 4ms/step - loss: 0.5296 - accuracy: 0.7250 - val_loss: 0.3784 - val_accuracy: 0.9468 Epoch 55/80 79/79 [=====] - 6s 4ms/step - loss: 0.5305 - accuracy: 0.7262 - val_loss: 0.3337 - val_accuracy: 0.9582 Epoch 56/80 79/79 [=====] - 6s 3ms/step - loss: 0.5288 - accuracy: 0.7250 - val_loss: 0.3246 - val_accuracy: 0.9582 Epoch 57/80 79/79 [=====] - 6s 3ms/step - loss: 0.5242 - accuracy: 0.7364 - val_loss: 0.2906 - val_accuracy: 0.9438 Epoch 58/80 79/79 [=====] - 6s 3ms/step - loss: 0.5261 - accuracy: 0.7338 - val_loss: 0.3031 - val_accuracy: 0.9438 Epoch 59/80 79/79 [=====] - 6s 3ms/step - loss: 0.5229 - accuracy: 0.7376 - val_loss: 0.3111 - val_accuracy: 0.9582 Epoch 60/80 79/79 [=====] - 6s 3ms/step - loss: 0.5203 - accuracy: 0.7326 - val_loss: 0.3103 - val_accuracy: 0.9438 Epoch 61/80 79/79 [=====] - 6s 3ms/step - loss: 0.5193 - accuracy: 0.7351 - val_loss: 0.3116 - val_accuracy: 0.9544 Epoch 62/80 79/79 [=====] - 6s 3ms/step - loss: 0.5179 - accuracy: 0.7364 - val_loss: 0.3067 - val_accuracy: 0.9506 Epoch 63/80 79/79 [=====] - 6s 3ms/step - loss: 0.5203 - accuracy: 0.7258 - val_loss: 0.3285 - val_accuracy: 0.9316 Epoch 64/80 79/79 [=====] - 6s 3ms/step - loss: 0.5176 - accuracy: 0.7414 - val_loss: 0.2669 - val_accuracy: 0.9586 Epoch 65/80 79/79 [=====] - 6s 3ms/step - loss: 0.5161 - accuracy: 0.7351 - val_loss: 0.2908 - val_accuracy: 0.9438 Epoch 66/80 79/79 [=====] - 6s 3ms/step - loss: 0.5181 - accuracy: 0.7465 - val_loss: 0.3235 - val_accuracy: 0.9628 Epoch 67/80 79/79 [=====] - 6s 3ms/step - loss: 0.5138 - accuracy: 0.7389 - val_loss: 0.2717 - val_accuracy: 0.9586 Epoch 68/80 79/79 [=====] - 6s 3ms/step - loss: 0.5181 - accuracy: 0.7498 - val_loss: 0.2722 - val_accuracy: 0.9586 Epoch 69/80 79/79 [=====] - 6s 3ms/step - loss: 0.5088 - accuracy: 0.7452 - val_loss: 0.2883 - val_accuracy: 0.9438 Epoch 70/80 79/79 [=====] - 6s 3ms/step - loss: 0.5058 - accuracy: 0.7529 - val_loss: 0.2887 - val_accuracy: 0.9582 Epoch 71/80 훈련 횟수(epochs)는 80회, batch_size는 10, validation_split은 0.25로 설정해 학습을 시켰다.
9. 검증 그래프 시각화	matplotlib을 이용해 훈련 횟수 대비 훈련 손실값(loss)과 검증 손실값(val_loss), 훈련 정확도(accuracy)와 검증 정확도(val_accuracy)를 나타내는 그래프를 만들었다.  <오차율(왼쪽)과 정확도(오른쪽) 시각화> ※첫번째 그래프에서 EPOCH(학습 횟수)가 늘어남에 따라 LOSS(실제 정답과 모델의 예측값의 차)가 줄어드는 것을 볼 수 있다.
10. 딥러닝 정확도 확인	<pre>#모델 검증 평가 model.evaluate(test_x, test_y)</pre> 15/15 [=====] - 0s 2ms/step - loss: 0.3098 - accuracy: 0.8936 [0.30976033210754395, 0.893569827079773] 분리된 데이터로 딥러닝 모델을 테스트 해본 결과, 정확도가 89%, 오차가 0.3이 나왔다.
11. 과적합 방지	딥러닝 모델에서 층(layer) 수를 늘릴수록 훈련 데이터에서만 높은 결과를 얻고, 테스트 데이터에서는 낮은 성능을 보이는 과대적합(overfitting) 문제를

방지하기 위해 **Dropout**를 이용하여 임의로 퍼셉트론 간의 연결(가중치)을 제거한다.



<과대적합>



<드롭아웃 : 가중치 제거>

12. 드롭 아웃과 RMS

#딥러닝 모델 구성하기

```
model3 = Sequential()
model3.add(Dense(30, input_dim=9, activation='relu'))
model3.add(tf.keras.layers.Dropout(0.1))
model3.add(Dense(1, activation='sigmoid'))
```

딥러닝 모델에서 층(layer) 수를 늘릴수록 훈련 데이터에서만 높은 결과를 얻고, 테스트 데이터에서는 낮은 성능을 보이는 과대적합(overfitting) 문제를 방지하기 위해 **Dropout**을 이용하여 임의로 퍼셉트론 간의 연결(가중치)을 제거한다.

```
#모델 컴파일
model4.compile(loss='binary_crossentropy', optimizer='rmsprop', metrics=['accuracy'])
model4.summary()
```

Model: "sequential_2"

Layer (type)	Output Shape	Param #
dense_4 (Dense)	(None, 30)	300
dense_5 (Dense)	(None, 1)	31

=====
Total params: 331
Trainable params: 331
Non-trainable params: 0

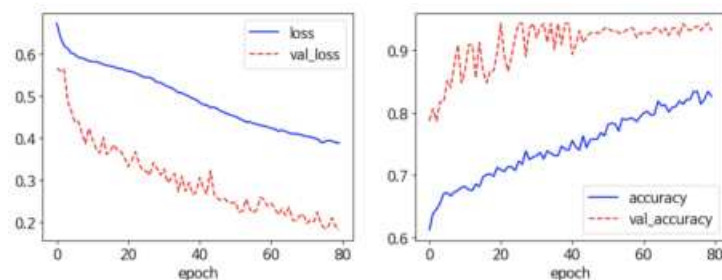
손실 함수에는 이진 분류에 사용되는 `binary_crossentropy`를 적용했으며, `optimizer` 즉, 최적화 알고리즘은 `RMS`를 사용하였다. 그리고 `metrics`에는 정확히 예측한 비율을 나타내는 `accuracy`를 이용했다.

- `RMS`는 평균제곱오차로 값을 최적화시켜주는 알고리즘중 하나이다.

```
#모델 학습
history = model4.fit(train_x, train_y, epochs=80, batch_size=10, validation_split=0.25)
```

79/79 [====] - 0s 3ms/step - loss: 0.4513 - accuracy: 0.7887 - val_loss: 0.2475 - val_accuracy: 0.9354
Epoch 52/80
79/79 [====] - 0s 3ms/step - loss: 0.4463 - accuracy: 0.7833 - val_loss: 0.2233 - val_accuracy: 0.9316
Epoch 53/80
79/79 [====] - 0s 3ms/step - loss: 0.4432 - accuracy: 0.7828 - val_loss: 0.2265 - val_accuracy: 0.9316
Epoch 54/80
79/79 [====] - 0s 3ms/step - loss: 0.4378 - accuracy: 0.7786 - val_loss: 0.2545 - val_accuracy: 0.9278
Epoch 55/80
79/79 [====] - 0s 3ms/step - loss: 0.4384 - accuracy: 0.7909 - val_loss: 0.2318 - val_accuracy: 0.9278
Epoch 56/80
79/79 [====] - 0s 3ms/step - loss: 0.4352 - accuracy: 0.7883 - val_loss: 0.2213 - val_accuracy: 0.9316
Epoch 57/80
79/79 [====] - 0s 3ms/step - loss: 0.4326 - accuracy: 0.7909 - val_loss: 0.2388 - val_accuracy: 0.9354
Epoch 58/80
79/79 [====] - 0s 3ms/step - loss: 0.4383 - accuracy: 0.7909 - val_loss: 0.2587 - val_accuracy: 0.9354
Epoch 59/80
79/79 [====] - 0s 3ms/step - loss: 0.4289 - accuracy: 0.7858 - val_loss: 0.2533 - val_accuracy: 0.9282
Epoch 60/80
79/79 [====] - 0s 4ms/step - loss: 0.4265 - accuracy: 0.7934 - val_loss: 0.2379 - val_accuracy: 0.9278
Epoch 61/80
79/79 [====] - 0s 5ms/step - loss: 0.4233 - accuracy: 0.7997 - val_loss: 0.2446 - val_accuracy: 0.9278
Epoch 62/80
79/79 [====] - 2s 21ms/step - loss: 0.4217 - accuracy: 0.8023 - val_loss: 0.2335 - val_accuracy: 0.9278
Epoch 63/80
79/79 [====] - 0s 5ms/step - loss: 0.4168 - accuracy: 0.7959 - val_loss: 0.2174 - val_accuracy: 0.9278
Epoch 64/80
79/79 [====] - 0s 5ms/step - loss: 0.4198 - accuracy: 0.7972 - val_loss: 0.2312 - val_accuracy: 0.9354
Epoch 65/80
79/79 [====] - 0s 3ms/step - loss: 0.4161 - accuracy: 0.8188 - val_loss: 0.2159 - val_accuracy: 0.9354
Epoch 66/80
79/79 [====] - 0s 3ms/step - loss: 0.4119 - accuracy: 0.8112 - val_loss: 0.2352 - val_accuracy: 0.9278
Epoch 67/80
79/79 [====] - 0s 3ms/step - loss: 0.4183 - accuracy: 0.8124 - val_loss: 0.2844 - val_accuracy: 0.9354
Epoch 68/80
79/79 [====] - 0s 3ms/step - loss: 0.4183 - accuracy: 0.8018 - val_loss: 0.2834 - val_accuracy: 0.9248
Epoch 69/80
79/79 [====] - 0s 3ms/step - loss: 0.4081 - accuracy: 0.8086 - val_loss: 0.2142 - val_accuracy: 0.9354
Epoch 70/80
79/79 [====] - 0s 3ms/step - loss: 0.4058 - accuracy: 0.8099 - val_loss: 0.2186 - val_accuracy: 0.9354
Epoch 71/80
79/79 [====] - 0s 3ms/step - loss: 0.4037 - accuracy: 0.8188 - val_loss: 0.1967 - val_accuracy: 0.9248
Epoch 72/80
79/79 [====] - 0s 4ms/step - loss: 0.4029 - accuracy: 0.8137 - val_loss: 0.2049 - val_accuracy: 0.9438

훈련 횟수(`epochs`)는 80회, `batch_size`는 10, `validation_split`은 0.25로 설정해 학습을 시켰다.



`matplotlib`을 통해 만든 그래프를 통해 훈련 횟수 대비 훈련 손실값(`loss`)과 검증 손실값(`val_loss`), 훈련

	정확도(accuracy)와 검증 정확도(val_accuracy)를 한눈으로 파악해준다. <pre>#모델 검증 평가 model4.evaluate(test_x, test_y)</pre> 15/15 [=====] - 0s 3ms/step - loss: 0.2329 - accuracy: 0.9069 [0.23293204605579376, 0.9068736433982849] 후에 드롭아웃을 통해 과대적합 문제를 해결하고 RMS로 최적화를 한 인공지능 모델을 만들었더니 오차는 0.23, 정확도는 90%로 손실값은 줄어 들고 정확도는 올라간 것을 확인할 수 있다.
13. 우리 데이터에 더 적합한 모델 선정	앞에서 만든 모델중 선형회귀는 정확도가 86%가 나왔고, 이후에 했던 딥러닝은 정확도가 89%에 오차 0.3, 드롭아웃으로 과적합 문제를 해결하고, RMS를 통해 값을 최적화시켜준 모델은 전보다 정확도가 올라가고 오차는 내려간 정확도가 90%에 오차는 0.23가 나오게 되었다. 따라서 우리는 이중 정확도가 가장 높고 오차가 가장 낮은 모델인 딥러닝에서 드롭아웃으로 과적합 문제를 해결하고, RMS를 통해 값을 최적화시켜준 모델이 더 적합하다는 것을 알 수 있다.
14. 평가 모델 선정	우리는 이중 정확도가 가장 높고 오차가 가장 낮은 모델인 딥러닝에서 드롭아웃으로 과적합 문제를 해결하고, RMS를 통해 값을 최적화시켜준 모델로 학생들의 우울증 여부를 판단해보기로 했다.

2.2.4. 직스 학생 및 주변 성인 대상 설문

앞서 만든 우울증 예측 프로그램을 통해 우리가 현재 살고 있는 인도네시아에서 살고 있는 한국인들 대상으로 실제 우울증을 예측해보기 위하여 설문을 진행해보았다. 설문은 구글에서 제공하는 설문지를 이용하였고, **SNS**나 온라인을 통해 이 설문을 배포했다. 설문의 질문은 총 **9**가지로 구성되어 있었고, 질문의 내용은 환자의 나이, 가끔 이유없는 슬픔을 느낄때가 있는지, 가족들에게 짜증을 내는지, 잠을 자는데 문제가 있는지, 집중을 잘 못하는지, 과식이나 적은 식욕을 가지고 있는지, 혹은 죄책감을 가지고 있는지, 가족과의 문제나 자살 시도에 대해 물어보고 있다. 이 질문의 내용은 데이터 수집과정에서

얻은 9개의 속성들을 물어보는 질문이다. 우리는 이 설문결과가 본 논문에 기재될 것임을 얘기하였고, 허락을 받고 이 결과를 논문에 사용하였다.

나이가 몇살이십니까? 내 답변	
가끔 이유없이 슬픈적이 있으십니까? ☐ 네 ☐ 아니오 ☐ 가끔	집중을 잘 못할 때가 있습니까? ☐ 네 ☐ 아니오 ☐ 가끔
가족들에게 짜증을 낼 때가 있습니까? ☐ 네 ☐ 아니오 ☐ 가끔	과식을 하거나 식욕이 없을 때가 있습니까? ☐ 네 ☐ 아니오 ☐ 가끔
잠을 자는데에 문제가 있으십니까? (ex) 불면증) ☐ 네 ☐ 아니오 ☐ 가끔	죄책감을 느낄 때가 있습니까? ☐ 네 ☐ 아니오 ☐ 가끔

집중을 잘 못할 때가 있습니까? ☐ 네 ☐ 아니오 ☐ 가끔
과식을 하거나 식욕이 없을 때가 있습니까? ☐ 네 ☐ 아니오 ☐ 가끔
죄책감을 느낄 때가 있습니까? ☐ 네 ☐ 아니오 ☐ 가끔

설문 통계 결과, 인도네시아 내 자카르타, 땅그랑, 찌까랑에 거주하고 있는 한인들과 JIKS에 재학중인 학생들 총 45명의 사람들이 이 설문에 응답해주었다. 이렇게 얻은 설문 결과 파일을 csv파일로 다운을 받은 후, 설문 응답에서 네는 1, 아니오는 0, 가끔은 2로 변환하여 컴퓨터가 설문 응답 값을 읽을 수 있도록 만들어주었다.

	우리가 설문조사를 통해 수집한 데이터를 URL을 통해 csv 파일 형식으로 읽어왔다.
2. 데이터 전처리 과정	설문의 응답이 수집된 시간인 타임스탬프를 제외시킨 후, 앞에서 학습에 쓰인 데이터와 동일하게 ‘네’를 1, ‘아니오’를 0, 가꿈을 2로 변경했다. <pre>df_s = df_s.drop(columns='타임스탬프') #응답의 형태 df_s.columns = ['나이', '출근 교통 수단', '가족들에게 짜증을 낸', '불안증', '집중을 잘 못함', '괴식 또는 적은 식욕', '자책감', '가족과의 문제', '자살 시도'] #네: 1, (아니오): 0으로 바꾸기 df_s = df_s.replace({'네':1, '아니오':0, '가꿈':2}) df_s.info()</pre> <pre><class 'pandas.core.frame.DataFrame'> RangeIndex: 45 entries, 0 to 44 Data columns (total 9 columns): # Column Non-Null Count Dtype --- -- 0 나이 45 non-null int64 1 출근 교통 수단 45 non-null int64 2 가족들에게 짜증을 낸 45 non-null int64 3 불안증 45 non-null int64 4 집중을 잘 못함 45 non-null int64 5 괴식 또는 적은 식욕 45 non-null int64 6 자책감 45 non-null int64 7 가족과의 문제 45 non-null int64 8 자살 시도 45 non-null int64 dtypes: int64(9) memory usage: 3.3 KB</pre> 위의 사진을 보면, 데이터 타입이 전부 int로, 우울증 예측 모델이 적용시킬 수 있는 형태가 되었다.
3. AI모델에 적용하기	우선, 설문조사 결과 데이터셋인 df_s를 numpy 배열로 바꿨다. 그 후, predict 함수를 이용해 우울증에 걸렸을 확률을 구했다.

```
test_x3 = df_s.to_numpy()
```

```
y_predict2 = model4.predict(test_x3)
y_predict2
```

```
2/2 [=====] - 0s 7ms/step
```

```
array([[0.44465125],
       [0.48081815],
       [0.5824583 ],
       [0.944016  ],
       [0.3453199 ],
       [0.43920392],
       [0.6859532 ],
       [0.3854341 ],
       [0.3211313 ],
       [0.7610984 ],
       [0.8894326 ],
       [0.41248265],
       [0.36412528],
       [0.7248084 ],
       [0.57657164],
       [0.8536387 ],
       [0.8486971 ],
       [0.6758023 ],
       [0.943829  ],
       [0.41248265],
       [0.8507475 ],
       [0.6880173 ],
       [0.97303027],
       [0.50341636],
       [0.5213376 ],
       [0.7723113 ],
       [0.13962217],
       [0.53383046],
       [0.79446834],
       [0.48244548],
       [0.45240465],
       [0.4022191 ],
       [0.94742376],
       [0.9377347 ],
       [0.7164742 ],
       [0.40655166],
       [0.7795976 ],
       [0.27967504],
       [0.9164037 ],
       [0.79907405],
```

그리고, 그 값이 0.5보다 크면 1, 크지 않으면 0으로 값이 바뀌도록했고, 결과는 다음과 같다.

```
y_predict2 = np.where(y_predict2 > 0.5, 1, 0)
y_predict2 = y_predict2.tolist()
for i, L in enumerate(y_predict2):
    print('{}번째 환자: {}'.format(i+1, L))
print('우울증 위험 환자수/총 환자수: {}/{}'.format(y_predict2.count([1]), len(y_predict2)))
```

	1번째 환자: [0]
	2번째 환자: [0]
	3번째 환자: [1]
	4번째 환자: [1]
	5번째 환자: [0]
	6번째 환자: [0]
	7번째 환자: [1]
	8번째 환자: [0]
	9번째 환자: [0]
	10번째 환자: [1]
	11번째 환자: [1]
	12번째 환자: [0]
	13번째 환자: [0]
	14번째 환자: [1]
	15번째 환자: [1]
	16번째 환자: [1]
	17번째 환자: [1]
	18번째 환자: [1]
	19번째 환자: [1]
	20번째 환자: [0]
	21번째 환자: [1]
	22번째 환자: [1]
	23번째 환자: [1]
	24번째 환자: [1]
	25번째 환자: [1]
	26번째 환자: [1]
	27번째 환자: [0]
	28번째 환자: [1]
	29번째 환자: [1]
	30번째 환자: [0]
	31번째 환자: [0]
	32번째 환자: [0]
	33번째 환자: [1]
	34번째 환자: [1]
	35번째 환자: [1]
	36번째 환자: [0]
	37번째 환자: [1]
	38번째 환자: [0]
	39번째 환자: [1]
	40번째 환자: [1]
	41번째 환자: [1]
	42번째 환자: [1]
	43번째 환자: [1]
	44번째 환자: [0]
	45번째 환자: [1]
	우울증 위험 환자수/총 환자수: 29/45

	총 45명의 설문조사 응답자 중, 29명이 우울증에 걸릴 위험이 있다는 결과가 나왔다.						
4. 위험성 각인 및 예방 방법 알리기	설문 조사를 했는 곳에서 우울증에 대한 위험성을 각인시켜주고, 예방 방법을 알려주었다.						
5. JIKS학생들과 인도네시아 내의 한인 우울증 환자 수 비교	총 45명의 설문조사 응답자중 JIKS학생들이 17명, 나머지 다양한 연령대의 인도네시아의 한인들이 28명으로 나타났다. JIKS학생들 17명중 11명인 64.7%가 우울증에 걸렸고, 나머지 연령대의 인도네시아의 한인들이 28명중 18명인 64.2%가 우울증에 걸린 것으로 나타났다. JIKS와 한인들의 우울증 환자 비율 <table border="1"> <thead> <tr> <th>그룹</th> <th>우울증 환자 비율 (%)</th> </tr> </thead> <tbody> <tr> <td>JIKS 학생들</td> <td>64.7</td> </tr> <tr> <td>인도네시아 내 한인들</td> <td>64.2</td> </tr> </tbody> </table> 둘의 차이는 그렇게 심하게 나지는 않지만 JIKS 학생들이 우울증 환자 비율이 더 높은 것으로 나타났다.	그룹	우울증 환자 비율 (%)	JIKS 학생들	64.7	인도네시아 내 한인들	64.2
그룹	우울증 환자 비율 (%)						
JIKS 학생들	64.7						
인도네시아 내 한인들	64.2						
6. JIKS학생들 대상 우울증 앙케이트	JIKS학생들이 얼마나 우울증에 대해 잘 알고 있는지에 대해 앙케이트를 실시하였다.						



	그 결과 잘 알고 있다가 8명, 잘 모르겠다가 9명으로 비슷하게 나타났지만 JIKS학생들중 우울증에 대해 모르는 학생들이 꽤나 있음을 알 수 있다.
5. 웹사이트 만들기	

7. 캠페인 활동



JIKS에서 캠페인 활동을 통해 학생들에게 우울증에 대해 경각심을 심어주었다.

2.3. 연구결과

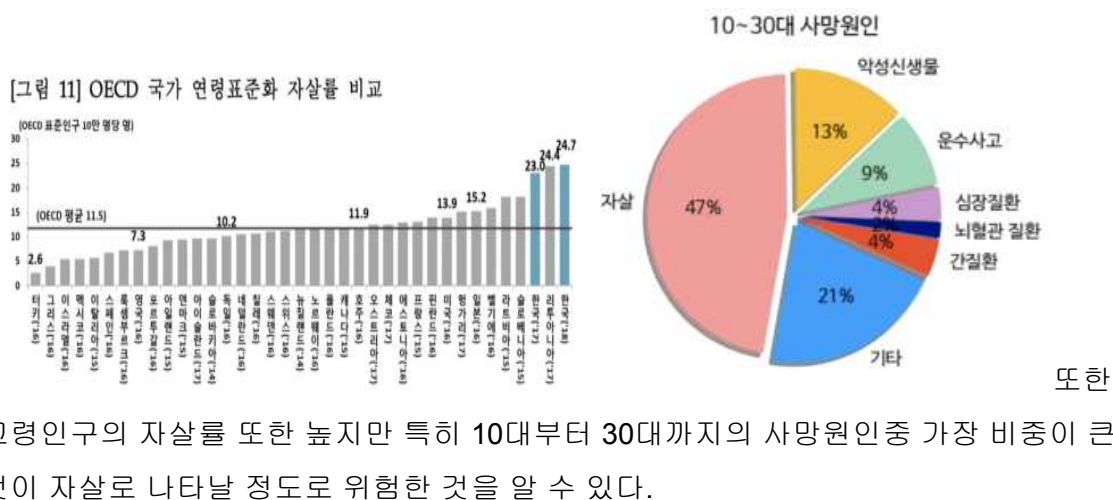
총 1503명의 우울증 환자들의 데이터를 가지고 우울증 예측 모델을 제작한 결과, 선형회귀에서는 정확도가 86%, 딥러닝에서는 정확도가 89%에 오차가 0.3이 나왔으나 드롭아웃으로 과적합 문제를 해결하고, 최적화 알고리즘을 adam대신 RMS를 통해 값을 최적화시켜준 후, 두 모델의 결과를 보니, 드롭아웃과 RMS를 하여 만든 모델은 정확도가 90%에 오차가 0.23으로 정확도가 올라가고, 오차는 줄어들었다. 그중 가장 정확도가 높았던 딥러닝에서 RMS으로 값을 최적화해주고, 드롭아웃으로 과적합 문제를 해결한 AI모델을 최종적으로 우울증 예측 모델로 제작하였다. 그리고 나는 한국을 떠나 인도네시아로 와서 지내고 있는 한국인들과 자카르타 한국 국제학교 학생들을 대상으로

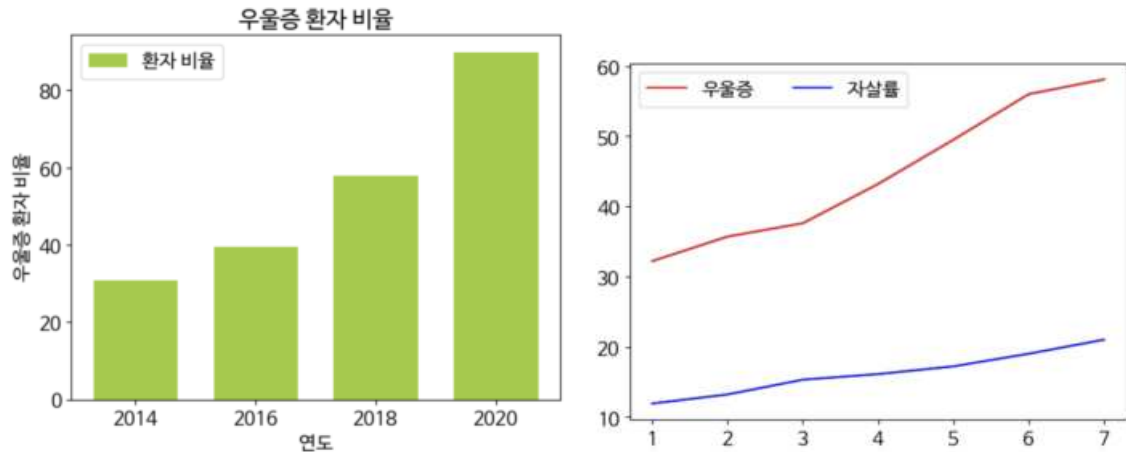
설문조사를 진행했고, 총 45명이 설문에 응답해주었다. 설문 결과를 모델에 적용할 수 있도록 변형시킨 후 그 값들을 대입하여 우울증을 예측한 결과, 45인의 JIKS학생들과 인도네시아 한인들 중 29명이 우울증 위험 결과가 나왔다. 그래서 우리는 이러한 우울증 위험 결과가 나온 사람들에게 우울증의 위험성을 다시 한 번 각인 시켜주고, 우울증 예방 방법을 알려주었다. 또한 45명의 설문대상자중 JIKS학생들이 17명, 나머지 다양한 연령대의 인도네시아의 한인들이 28명으로 나타났다. 그리고 JIKS학생들 17명중 11명인 64.7%가 우울증에 걸렸고, 나머지 연령대의 인도네시아의 한인들이 28명중 18명인 64.2%가 우울증에 걸린 것으로 나타났다. 이를 통해 둘의 차이는 그렇게 심하게 나지는 않지만 JIKS 학생들이 우울증 환자 비율이 더 높은 것을 알 수 있었다. 이러한 이유는 JIKS학생들 17명의 응답자 전원이 12학년이였었고, 그들을 대상으로 몇몇한테 질문을 해본 결과 많은 과제와 입시 스트레스 때문으로 나타났다. 이후 JIKS학생들이 얼마나 우울증에 대해 잘 알고 있는지에 대해 의문이 생겨 앙케이트를 실시한 결과, JIKS학생들중 많은 수의 학생이 우울증에 대해 잘 모르는 것으로 나타났다. 그래서 웹사이트와 포스터를 만들어 JIKS에서 캠페인 활동을 하며 JIKS 학생들에게 우울증에 대한 경각심을 심어주었다. 그리고, 우울증과 9개의 속성들간의 상관관계를 히트맵으로 나타낸 결과는 집중을 잘 못함, 가족과의 문제가 있음, 자살 시도의 순서대로 높은 연관성을 보였다.

3. 본론

3.1. 연구 요약 및 의의

해가 지나면 지날수록 국내에서는 점점 자살을 하는 사람들이 늘어나고 있다. 보건복지부의 통계에 따르면, 자살사망자의 수가 1.2%나 증가했다는 것을 알 수 있었고, 또한 통계청의 조사에 따르면 특히 대한민국은 OECD 국가들중 자살률이 총 1등인 것으로 나타날 정도로 한국에서는 자살 문제가 다른 나라에 비해 특히 심각한 것으로 알 수 있다.



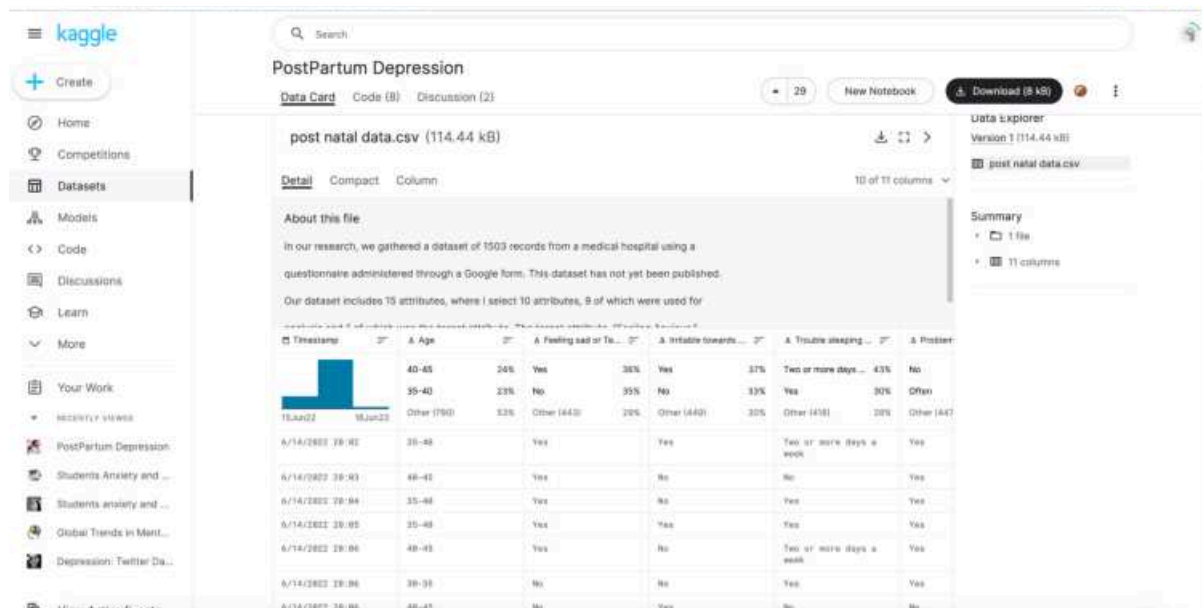


그리고 자살에 관한 이유는 보건복지부의 조사에 따르면 정신건강 문제라는 것을 밝혀내었는데 그중 우울증이 제일 연관성이 높았고, 자살률과 함께 점점 수가 많이 늘고 있는 것으로 나타났다. 또한 우울증은 시간이 지날수록 자살할 확률과 함께 심화되어져 가는 것으로 나타나는데, 초기에는 우울증을 알게 되면 간단한 치료를 통해 해결될 수 있지만, 이러한 우울증을 계속 방치하게 되면 끝내 극단적인 선택을 할 수 있다. 그렇기에 우울증은 초기에 빠르게 발병 여부를 확인하고 치료하는 것이 중요하다.

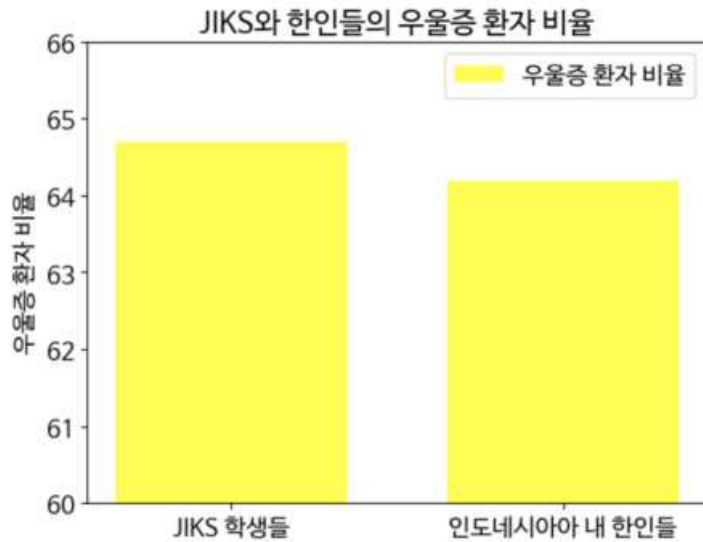


하지만 이러한 우울증을 검사하기 위해선 병원에 가서 객관적인 검사를 받아야하지만, 우리나라에서는 대다수의 사람들이 자기가 우울증이 걸린 것을 부인하고, 그것이 단순히 사춘기나 자신의 성격 때문이라고 여기거나 정신건강의학과에 가는 것을 꺼리거나 스스로 병을 해결할 수 있다고 생각하여 병원에 가서 치료를 받으려고 하지 않는다. 우울증이 의심되면 우선 우울증을 일으킬 수 있는 질환에 대해 감별 진단을 하여 갑상선 기능 저하증 등의 내분비 질환과 같은 내과 질환에 대한 감별이 필요하고, 뇌졸중과 같은 신경과적 문제에서도 우울증이 발생할 수 있다는 것을 정밀검사를 통해 확인하고, 다양한 면담이나 DSM-IV와 같은 진단을 한다거나 이차적 우울증을 감별하기 위해 혈액화적 검사 및 뇌영상 검사등을 시행하여 우울증의 진행 정도를 판단하는데, 대부분의 사람들은 이러한 과정의

복잡성과 비용 문제, 병원에 가기 귀찮다는 등의 이유로 우울증을 초기에 알아내어 치료하지 못하고, 생활을 하다가 우울증에 걸리고 많은 시간이 지나서야 치료를 받거나 치료를 받지 못해 극단적인 선택을 할 수도 있다. 그렇기 때문에 나는 우울증에 대한 병의 인지율을 높이고, 우울증을 초기에 알아내서 예방하자는 목적을 가지고, 자신이 우울증의 증상을 가지고 있을 때, 병원에 가거나 복잡한 절차를 하지 않고, 간편하게 자신의 우울증 의심을 예측해볼 수 있는 우울증 예측 모델을 제안했다.



이 연구는 **1503명**의 우울증 환자 데이터를 가지고 인공지능을 만들었다. 보다 정확도가 높은 인공지능을 만들기 위해 선형회귀와 딥러닝 두가지 모델을 통해 나의 데이터에 더 적합한 모델을 찾기 위해 두가지 모델로 인공지능을 만들었다. 선형회귀에서는 정확도가 **86%**, 딥러닝에서는 정확도가 **89%**에 오차가 **0.3**이 나왔으나 더욱 딥러닝 모델의 정확도를 올리고 정교하게 만들어주기 위해 드롭아웃으로 과적합 문제를 해결하고, 최적화 알고리즘을 **adam**대신 **RMS**를 통해 값을 최적화시켜준 후, 두 모델의 결과를 보니, 드롭아웃과 **RMS**를 하여 만든 모델은 정확도가 **90%**에 오차가 **0.23**으로 정확도가 올라가고, 오차는 줄어들었다. 그중 가장 정확도가 높았던 **RMS**으로 값을 최적화해주고, 드롭아웃으로 과적합 문제를 해결한 **AI**모델을 최종적으로 우울증 예측 모델로 제작하였다. 또한 나는 한국을 벗어나 인도네시아에 거주중인 한국인들과 자카르타 한국 국제학교 학생들을 대상으로 설문조사를 진행했고, 총 **45명**이 설문에 응답해주었는데, 이 설문 결과를 모델에 적용할 수 있도록 변형시킨 후 그 값들을 대입하여 우울증을 예측한 결과, **45인**의 인도네시아 한국인들 중 **29명**이 우울증 위험 결과가 나왔다. 그래서 나는 이러한 우울증 위험 결과가 나온 사람들에게 우울증의 위험성을 다시 한 번 각인시켜주고, 우울증 예방 방법을 알려주었다.



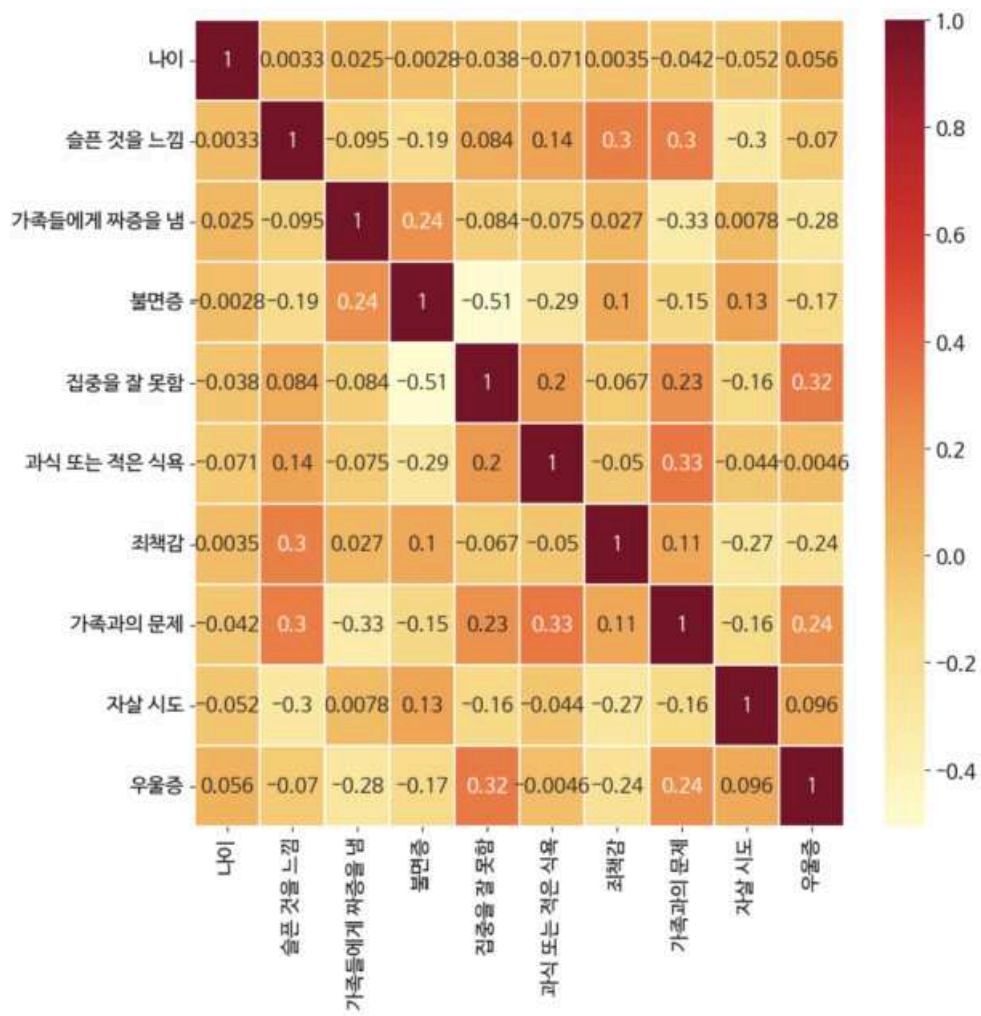
또한 45명의 설문대상자중 JIKS학생들이 17명, 나머지 다양한 연령대의 인도네시아의 한인들이 28명으로 나타났는데 JIKS학생들 17명중 11명인 64.7%가 우울증에 걸렸고, 나머지 연령대의 인도네시아의 한인들이 28명중 18명인 64.2%가 우울증에 걸린 것으로 나타났다. 이를 통해 둘의 차이는 그렇게 심하게 나지는 않지만 JIKS 학생들이 우울증 환자 비율이 더 높은 것을 알 수 있었다. 이러한 이유는 JIKS학생들 17명의 응답자 전원이 12학년이였었고, 그들을 대상으로 몇몇한테 질문을 해본 결과 많은 과제와 입시 스트레스 때문으로 나타났다.



이후 JIKS학생들이 얼마나 우울증에 대해 잘 알고 있는지에 대해 의문이 생겨 양케이트를 실시한 결과, JIKS학생들중 많은 수의 학생이 우울증에 대해 잘 모르는 것으로 나타났다.



그리고 우울증에 대한 웹사이트와 포스터를 만들고, 그것을 활용하여 JIKS에서 캠페인 활동을 진행하여 JIKS 학생들에게 우울증에 대한 경각심을 심어주었다.



또한, 우리가 수집한 우울증 환자 데이터를 통해 데이터를 분석하여 히트맵으로 이것을 나타냈을 때, 집중을 잘 못함, 가족과의 문제가 있음, 자살 시도가 우울증과 가장 상관관계가 높았는 것을 알아낼 수 있었다. 이러한 우울증 예측 AI 모델은 앞서 말했는 병원에 가는 수고와 비용적인 문제를 해결하여주고, 귀찮고, 복잡한 절차때문에 검사조차 받지 않았던 사람들이 간편히 몇초 만에 자신이 우울증 의심 환자인지를 알 수 있고, 여기서 우울증 의심 결과가 나오면, 바로 병원으로가서 더욱 정밀한 검사를 받을 수 있다. 이러한 점에서 본 논문에서 제작한 우울증 예측 AI 모델은 앞으로 우울증 진단 프로그램으로 누구나 손쉽게 사용할 수 있고, 사람들의 우울증을 복잡한 절차와 병원에 가는 일 없이 최대한 빠르고 손쉽게 진단해보고, 예방할 수 있다는 점에서 이 AI 모델은 큰 의미를 가진다.

4. 참고문헌

우울증 데이터 :

<https://www.kaggle.com/datasets/parvezalmuqtadir2348/postpartum-depression>

우울증 증가 :

<https://www.hira.or.kr/bbsDummy.do?pgmid=HIRAA020041000100&brdScnBltno=4&brdBltno=10627&pageIndex=1>

우울증 인공지능 :

<https://colab.research.google.com/drive/1mGlVW2dSfE9iKofXDugnLGNUZJjalfao?usp=sharing>

자살률 증가 :

https://www.mohw.go.kr/react/al/sal0301vw.jsp?PAR_MENU_ID=04&MENU_ID=0403&page=1&CONT_SEQ=373035

우울증 원인 : <https://www.yna.co.kr/view/AKR20180816166700797>

서울 아산 병원 우울증 :

<https://www.amc.seoul.kr/asan/healthinfo/disease/diseaseDetail.do?contentId=31581>

서울대학교 병원 우울증 :

<http://www.snuh.org/health/nMedInfo/nView.do?category=DIS&medid=AA000353>

sklearn 설명 : <https://engineer-mole.tistory.com/16>

딥러닝 비용함수 설명 :

<https://yeong-jin-data-blog.tistory.com/entry/%EB%94%A5%EB%9F%AC%EB%8B%9D-%EA%B0%9C%EB%85%90-%EB%B9%84%EC%9A%A9%ED%95%A8%EC%88%98-Cost-Function>

서울 삼성병원 우울증과 자살 :

http://www.samsunghospital.com/dept/medical/healthSub02View.do?content_id=1661&cPage=1&DP_CODE=DEP&MENU_ID=004025&ds_code=D0001673&main_content_id=406

자살률 증가 : <https://www.hani.co.kr/arti/society/health/1046946.html>

OECD 자살률 순위:

https://ko.wikipedia.org/wiki/OECD_%ED%9A%8C%EC%9B%90%EA%B5%AD_%EC%9E%90%EC%82%B4%EB%A5%A0_%EB%AA%A9%EB%A1%9D

삼성병원 우울증 예방 :

http://www.samsunghospital.com/dept/medical/healthSub01View.do?content_id=859&DP_CODE=DEP&MENU_ID=004025&ds_code=D0001673&main_content_id=858

블로그 인용한 글 :

<https://m.blog.naver.com/PostView.naver?isHttpsRedirect=true&blogId=iammaum&logNo=221221756690>

RMS : <https://m.blog.naver.com/pkw00/220226903866>

드롭아웃 : <https://inuplace.tistory.com/564>

과적합 : <https://untitledblog.tistory.com/68>